INTRODUCTION TO DATA SCIENCE

FINAL COMPILED ASSIGNMENT

Predicting Seven-Day Municipal Service-Request Non-Completion Risk

Questions 1-8 with separate answers, code, figures and evidence

PROJECT FIELD	DETAIL
Module	IDA117V - Introduction to Data Science
Student	Thapelo Monakedi
Student number	[INSERT STUDENT NUMBER]
Group / lecturer	[INSERT GROUP NUMBER AND LECTURER]
Other group members	[INSERT VERIFIED NAMES AND STUDENT NUMBERS]
Submission date	[INSERT SUBMISSION DATE]

DOCUMENT NOTE

This document compiles only the independently developed Question 1-8 answers and verified modelling artifacts created before later external drafts were introduced. No later draft was used as a source. Existing unsupported items remain explicitly marked as assumptions or placeholders.

BEFORE SUBMISSION

Replace only the clearly marked student, group, repository, licence, approval, demonstration-link and contribution placeholders with verified evidence. Do not invent approvals, links, signatures or future service outcomes.

Document map

SECTION	SUB-QUESTIONS	CONTENT
Question 1	1.1-1.3	Problem, stakeholder, target, metrics, data and ethics
Question 2	2.1-2.4	Data dictionary, audit, EDA, split and pipeline
Question 3	3.1-3.5	Baseline, two traditional models, two neural models and rationale
Question 4	4.1-4.4	Metrics, reports, confusion matrices, curves, comparison and overfitting
Question 5	5.1-5.3	Champion optimisation and locked final test
Question 6	6.1-6.2	Ablation and McNemar validation
Question 7	7.1-7.4	User-interface use and demonstration
Question 8	8.1-8.3	Recommendation, reproducibility and team defence

Evidence status

- All dataset and model numbers come from the completed code runs and saved results.
- Question 1 clearly marks facts, assumptions and placeholders.
- Question 3 records the entity-embedding network as the implemented second tabular neural architecture. Lecturer approval is required by the template, but no approval evidence has been supplied; approval is not claimed.
- Question 7 gives a tested local launch method and recorded evidence for three valid cases, one invalid case and an HTTP 200 health check. No public app link or presentation recording is claimed.
- Question 8 records the completed manual-upload repository package and its exact target URL. Public repository accessibility, final commit hash, dataset redistribution approval, public deployment link, presentation recording and signed contributions remain unverified or outstanding as stated.

INTRODUCTION TO DATA SCIENCE

QUESTION 1 - REVISED ANSWER

Problem, Stakeholder, Data and Ethics

Separate answers for Sub-questions 1.1, 1.2 and 1.3

EVIDENCE LABELS

VERIFIED FACT means directly computed from the supplied CSV or directly supported by the cited public source. ASSUMPTION means a project design choice that requires stakeholder confirmation. PLACEHOLDER means required evidence or approval that has not been supplied and must not be presented as complete.

Verified dataset snapshot

FIELD	CODE-VERIFIED VALUE
Exact file	Service_Requests_01_April_2026_until_30_May_2026 (1).csv
Rows / columns	247,795 / 14
Local creation coverage	1 April-30 May 2026 (60 days)
Within seven days	152,707 (61.6%)
At risk under project definition	95,088 (38.4%)
Missing completion field	73,606 (29.7%)

1.1 Service-delivery problem and classification task (4 marks)

WHAT THE QUESTION ASKS

Define the South African service-delivery problem, affected population, decision-maker and intended benefit. Convert it into a classification task by specifying prediction unit, inputs, target classes and horizon.

Answer to 1.1

Municipal service teams receive many requests, but some take longer than seven days to complete. Staff have limited time and resources. A risk score made when a request is logged can help staff review difficult cases earlier and plan work. The affected group is the people and organisations linked to requests in the dataset. The file has no demographic fields, so it cannot describe this group in more detail. The intended users are service managers, work-centre managers and triage staff. The score only gives advice; a human manager makes the final decision.

TASK ELEMENT	PROJECT DEFINITION
Prediction unit	One newly created service-request row.
Creation-time inputs	Sub-council, ward, suburb, complaint type, work centre, derived creation-date features, and coordinate-missing/zero indicators.

Target class 0	Completed between creation and the end of day 7, inclusive.
Target class 1	Completed after day 7 or no completion recorded in the extract.
Prediction horizon	At creation, predict the seven-calendar-day outcome.
Intended benefit	Add human attention to cases likely to remain unresolved; inform workload planning.
Do not use the score to	The score does not measure urgency, deservingness or legal service entitlement.

STATUS	STATEMENT	BASIS / ACTION
VERIFIED FACT	The extract has 247,795 rows created across 60 local dates, 1 April-30 May 2026.	Computed from Created_On_Date in the attached CSV.
VERIFIED FACT	95,088 rows (38.4%) meet the project at-risk definition.	Computed from Created_On_Date and Completed_Date.
ASSUMPTION	Seven calendar days is a useful cross-service review horizon.	Confirm against complaint-specific municipal service standards before use.
ASSUMPTION	Work_Center is available and stable at creation time.	Confirm source-system timestamp semantics with the data owner.
PLACEHOLDER	Named accountable stakeholder and approved operational decision.	Insert the real municipal owner/approver after written confirmation.

KEY POINT

The task is clear and measurable. Seven days and Work_Center availability at creation are still assumptions. The score may add attention only; it cannot remove service or close a request.

1.2 Technical metric, service indicator and error costs (2 marks)

WHAT THE QUESTION ASKS

State one primary technical metric and at least one service-delivery success indicator. Explain the cost of false positives and false negatives.

Answer to 1.2

The main metric is macro-F1 because it gives equal importance to both classes. Accuracy alone is not enough: the dummy model achieved 55.8% accuracy but found no at-risk requests. We also report precision, recall, PR-AUC, ROC-AUC, weighted-F1 and confusion matrices.

ITEM	DEFINITION / USE
Primary metric: macro-F1	Average of the two class F1 scores with equal class weight; used for validation-led selection.
Service indicator: top-k capture	Share of genuine at-risk cases captured when review capacity is limited to the top-ranked k% of requests.
Future service outcome	Change in overdue share or median completion time during a controlled pilot, without subgroup deterioration. PLACEHOLDER until a pilot occurs.

In the test data, reviewing the top 20% of scores found 40.2% of at-risk requests with 88.8% precision. This is based on past data and does not prove that live service delivery will improve.

ERROR	MEANING	POTENTIAL COST	SAFEGUARD

False positive	Flagged at risk but completed within seven days.	Unnecessary review; queue congestion; opportunity cost.	Capacity cap, ranked queue and human override.
False negative	Predicted timely but exceeded seven days.	Missed escalation and possible prolonged service disruption.	Emergency/criticality rules, repeat review and lower-threshold monitoring.

STATUS	STATEMENT	BASIS / ACTION
VERIFIED FACT	Top-20% test review captured 40.2% of at-risk cases at 88.8% precision.	Calculated from locked test probabilities and labels.
VERIFIED FACT	The selected champion had at-risk precision 84.5% and recall 46.9% at threshold 0.65.	Completed model evaluation.
ASSUMPTION	False negatives may be more costly for critical electricity, water, sanitation or safety cases.	Requires service-specific cost and criticality confirmation.
PLACEHOLDER	Prospective improvement in completion time or overdue share.	Measure during a real shadow/pilot period; do not claim now.

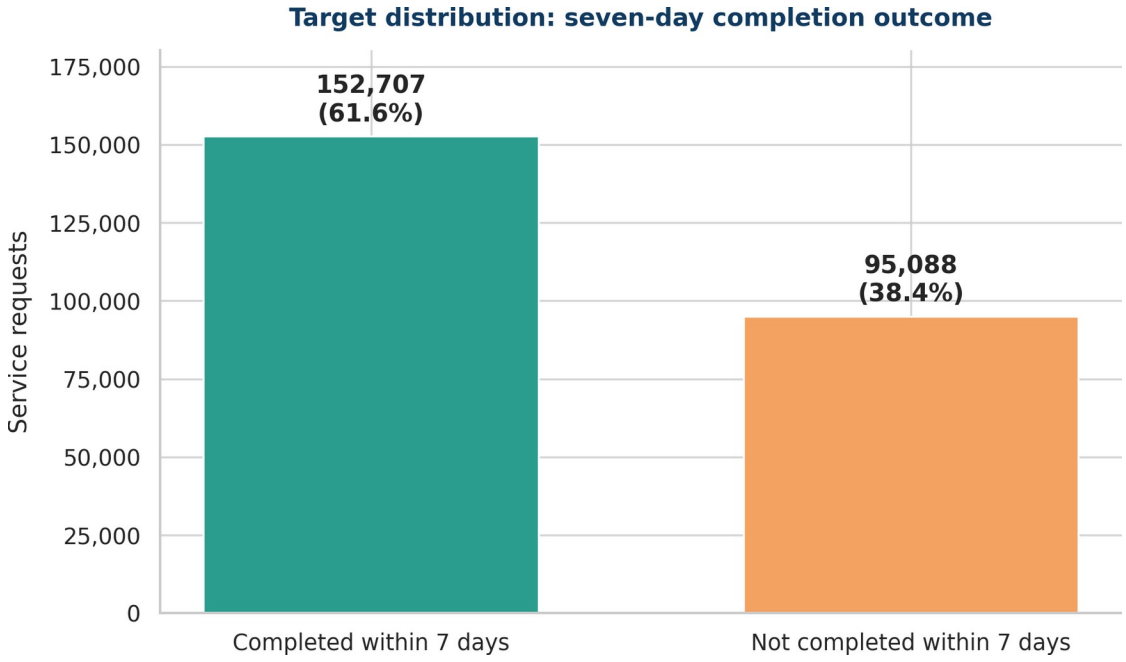


Figure 1.2. Code-verified target distribution in the supplied CSV.

KEY POINT

Macro-F1 is the main metric. Top-k capture shows how the model can fit a limited review queue. Live service improvement has not yet been tested.

1.3 Relevance, provenance, licence, coverage, limitations and safeguards (4 marks)

WHAT THE QUESTION ASKS

Justify dataset relevance, provenance, licence, coverage and limitations. Identify POPIA, privacy, fairness and misuse risks and propose safeguards.

Answer to 1.3 - relevance and coverage

The file is suitable because each row is a service request and includes the complaint type, work centre, location, creation date and completion date. The CSV has 247,795 rows and 14 columns. Requests were created from 1 April to 30 May 2026. It has 743 complaint types, 801 work centres, 118 ward labels and 4,711 cleaned suburb labels. Because it covers only two months, the results may not apply to other seasons or longer periods.

Provenance and licence - careful status

STATUS	STATEMENT	BASIS / ACTION
VERIFIED FACT	The supplied schema and first records match the City of Cape Town Open Data Portal service-request item identified in the project research.	Public page: “Service Requests 2023 until 10 September 2026”; it describes SAP C3 Notifications records and City of Cape Town Corporate GIS ownership.
RECORDED EVIDENCE - NOT INDEPENDENTLY ARCHIVED	The assignment research recorded that the portal item was displayed as Public with a “Custom License”.	Evidence status: statement recorded in the assignment; a dated screenshot/export and exact item page metadata are still required for the submission evidence folder.
VERIFIED FACT	City policy defines open data as data that can be freely used, shared and built on, subject to privacy/confidentiality governance.	City of Cape Town Open Data Policy 27781.
NOT EVIDENCED / APPROVAL NOT CLAIMED	Exact item-level Custom License text and permission to redistribute the supplied dataset.	Required evidence: archive the exact licence text and obtain written redistribution confirmation from the data owner or authorised source before publishing raw data.
NOT EVIDENCED	Written chain-of-custody confirmation that the supplied file is an unchanged export of the named portal item.	Required evidence: written confirmation from the lecturer or data owner. Until supplied, the raw dataset remains excluded from the public package.

Source links recorded for verification: City dataset page: https://odp-cctegis.opendata.arcgis.com/datasets/service-requests-2023-until-10-september-2026/explore ; City Open Data Portal: https://www.capetown.gov.za/city-connect/all-city-online-services/open-data-portal/ ; Open Data Policy: https://www.capetown.gov.za/media/vg2plaql/open_data_policy.pdf .

Data limitations and quality evidence

LIMITATION	VERIFIED EVIDENCE	EFFECT
Missing completion	73,606 rows (29.7%).	Blank completion is conservatively treated as at risk; status semantics must be confirmed.
Coordinate quality	About 33,575 values missing per coordinate field; 41,865 rows equal 0,0.	Raw coordinates excluded; only quality flags used.
Missing location	Sub-council 15,209; ward 9,882; suburb 5,184.	Missing category retained and monitored.
Short coverage	60 creation days.	Does not cover all seasons or long-term process changes.
No demographics	No demographic attributes appear in the 14 column names.	The CSV cannot directly test fairness by demographic group.

POPIA, privacy, fairness and misuse

The CSV has no fields named for a person, surname, email, phone number, contact details, street address or free-text description. This does not prove that the data is anonymous. Ward, suburb and coordinate fields could still identify a place when combined with other data. The responsible organisation must decide whether the processing meets POPIA requirements. This project does not make a legal-compliance claim.

RISK	WHY IT MATTERS	SAFEGUARD
Privacy / linkage	Location and coordinates may reveal or help infer a household/property.	Minimise location; omit raw coordinates; restrict row-level access; aggregate reporting.
Historical bias	Outcomes reflect infrastructure, routing and past resource allocation.	Audit error rates by service/location; retain human review; prohibit deservingness interpretations.
Observed subgroup performance	Test recall varied from 13.1% to 58.1% across 21 sub-council groups with n>=200.	Investigate gaps; monitor recall/FPR; block autonomous use.
Feedback loop	Prioritised cases may later make the model appear more accurate.	Log interventions; retain audit samples; separate natural outcomes from assisted outcomes.
Misuse	Low scores could be used to delay, close or deny requests.	Policy prohibition: score may add attention only; emergency rules override.
Automation bias	Operators may treat probability as certainty.	Display probability, limitations and explanations; require accountable human decision.

STATUS	STATEMENT	BASIS / ACTION
VERIFIED FACT	No column names directly indicate names, emails, phone numbers, addresses or free text.	Checked by code across all 14 source columns.
VERIFIED FACT	Sub-council at-risk recall ranged 13.1%-58.1% for groups with at least 200 test rows.	The subgroup results differ, but this alone does not prove unlawful discrimination.
ASSUMPTION	Sub-council, ward, suburb and coordinates may be quasi-identifying when linked.	Treat as privacy-sensitive unless the data owner confirms otherwise.
PLACEHOLDER	Formal POPIA lawful-basis, privacy-impact and retention approval.	Must be approved by the responsible organisation or information officer.
PLACEHOLDER	Demographic fairness assessment.	Cannot be done from this CSV alone. It needs lawful and suitable group data or an approved proxy method.

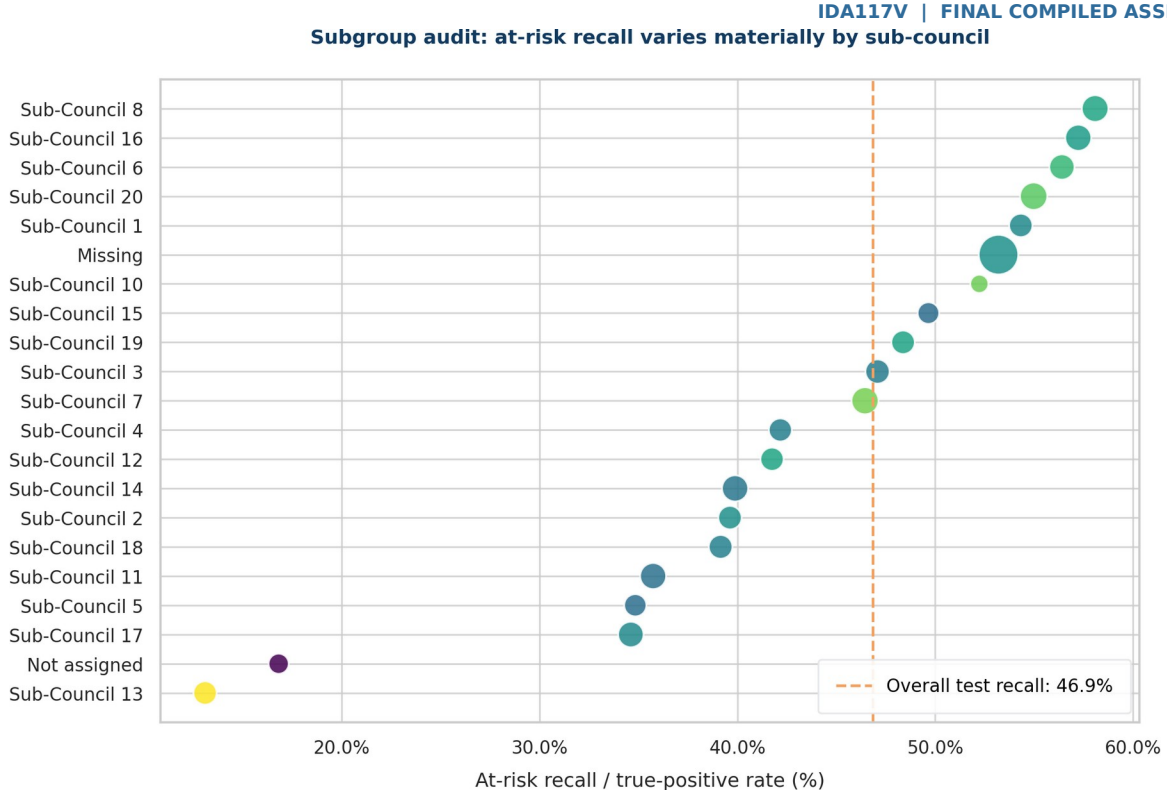


Figure 1.3. Verified subgroup recall variation by sub-council (n >= 200).

KEY POINT
Available evidence is limited to the assignment's recorded public-item metadata and policy references. The exact item-level licence text, chain of custody and written redistribution approval are not present, so approval is not claimed and the raw dataset is excluded. Privacy, fairness and legal-use checks also remain required.

The dataset is relevant and covers two verified months. Public metadata suggests a City of Cape Town SAP C3 source. Exact chain of custody and licence terms still need confirmation. Privacy, fairness and legal-use checks are also still required.

INTRODUCTION TO DATA SCIENCE

QUESTION 2 - FINAL COMPILED ANSWER

Data Audit, EDA, Split and Pipeline

Separate answers for Sub-questions 2.1, 2.2, 2.3 and 2.4

EVIDENCE BOUNDARY

This section uses the independently completed CSV audit, figures, split and modelling pipeline. Quantities are code-derived from the supplied service-request CSV. Unsupported use or provenance claims are not introduced here.

2.1 Data dictionary and audit (4 marks)

WHAT THE QUESTION ASKS

Create a data dictionary and audit class balance, duplicates, missingness, invalid values, outliers and possible leakage.

Answer to 2.1

FIELD	TYPE	MISSING	ROLE	INTERPRETATION
Sub_Council	Categorical	6.1%	Use	Administrative grouping; missing/odd labels monitored.
Ward	Categorical	4.0%	Use	Text preserves leading zeros.
Suburb	Categorical	2.1%	Use	High-cardinality place label.
C3_Complaint_Type	Categorical	0%	Use	Primary request category.
Work_Center	Categorical	0%	Use with caution	Must be available at creation.
Notification	Identifier	0%	Exclude	Near-unique ID.
Notification_type	Mixed text/ID	0%	Exclude	Many unique values and similar to an identifier.
Coordinates 1/2	Numeric	13.5%	Flags only	Raw values omitted; missing/zero flags used.
Created_On_Date	Datetime	0%	Derive	Local calendar features.
Changed_on	Datetime	3.5%	Exclude	Known after creation, so it would leak future information.
Completed_Date	Datetime	29.7%	Target only	Builds seven-day outcome.
Notifications_Created	Count	<0.01%	Exclude	Almost every row has the same value.
ObjectId	Identifier	0%	Exclude	Unique row key.
risk_7d	Binary derived	0%	Target	1 = not within seven days.

The dataset has 247,795 rows and 14 source columns. There are 152,707 timely requests (61.6%) and 95,088 at-risk requests (38.4%). There are no exact duplicates across all 14 columns. However, 730 rows match when ObjectId is ignored, and 3,020 rows use repeated Notification values. We kept them because they may be real repeat reports and we did not have a confirmed business key.

Main data-quality findings were 73,606 missing completion dates, about 33,565 missing values in each coordinate field, 41,865 coordinate pairs recorded as 0,0, 1,100 Sub_Council values recorded as 0, 6,929 Ward values recorded as 000, and 980 generic suburb labels. There were 8,072 completion times over 30 days, 7,106 over 90 days and 415 over 180 days. Some completion and change dates were later than the report date. We kept the original values and marked this as a limitation that needs confirmation.

Code used for 2.1

```
df = pd.read_csv(DATA, encoding="utf-8-sig", low_memory=False)
for column in ["Created_On_Date", "Changed_on", "Completed_Date"]:
    df[column] = pd.to_datetime(df[column], errors="coerce", utc=True)
duration_days = (
    (df["Completed_Date"] - df["Created_On_Date"]).dt.total_seconds()/86400
)
df["risk_7d"] = (~duration_days.between(0, 7, inclusive="both")).astype("int8")
missing = df.isna().mean().mul(100)
exact_duplicates = df.duplicated().sum()
duplicates_without_objectid = df.duplicated(
    subset=[c for c in df.columns if c != "ObjectId"]
).sum()
```

Question 2.1 - target, missingness and duplicate audit

2.2 Purposeful visualisations (6 marks)

> **WHAT THE QUESTION ASKS**
>
> Present at least five purposeful figures: target distribution, missingness/data quality, two feature-target relationships, and a geographic, temporal or subgroup view. Include titles, units, labels and interpretation.

Answer to 2.2

Seven figures were created: target balance, missingness, complaint-type risk, sub-council risk, daily volume and risk, completion time, and complaint volume versus risk. These figures support the metric choice, features, time-based split and subgroup monitoring.

Code used for 2.2

```
# Example: target distribution and complaint-type relationship.
sns.countplot(data=df, x="risk_7d")
plt.title("Target distribution: seven-day completion outcome")
plt.ylabel("Service requests")

complaint = df.groupby("C3_Complaint_Type").agg(
    requests=("ObjectId", "size"),
    risk_rate=("risk_7d", "mean")
).sort_values("requests", ascending=False).head(15)
complaint.sort_values("risk_rate").plot.barh(y="risk_rate")
plt.xlabel("Requests not completed within 7 days (%)")
```

Question 2.2 - representative target and feature-target plotting code

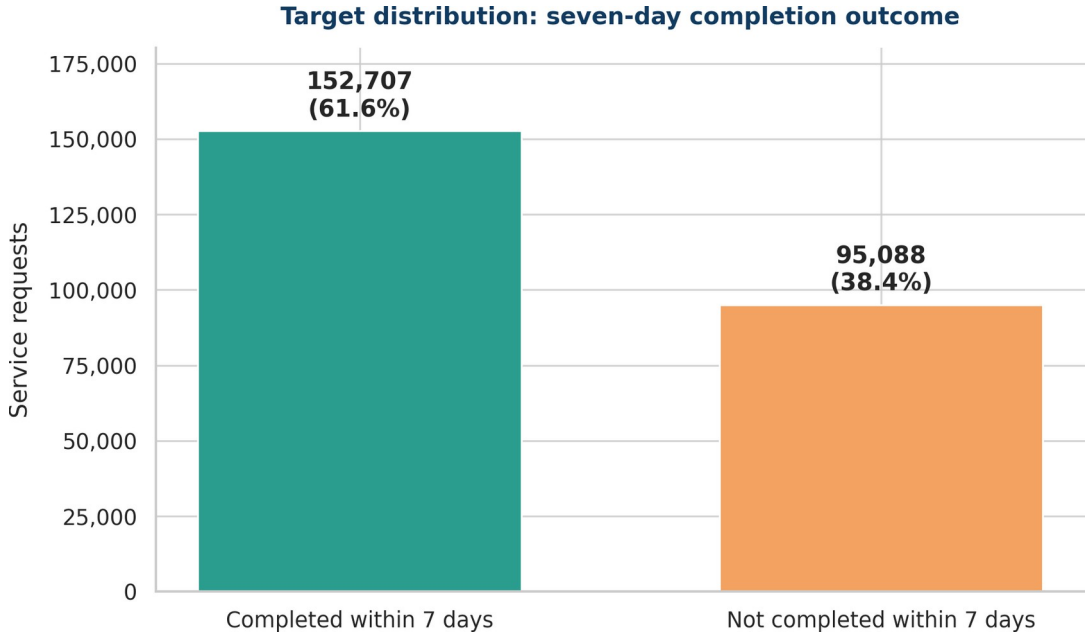


Figure 2.2A. Target distribution.

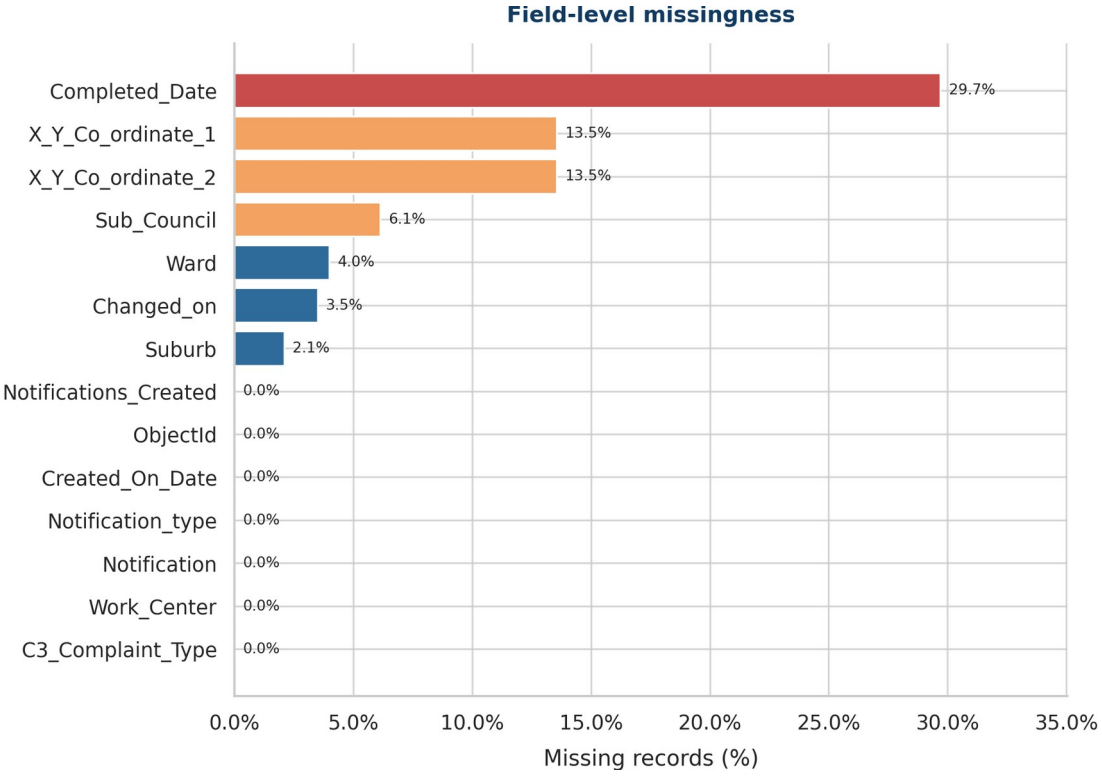


Figure 2.2B. Field-level missingness.

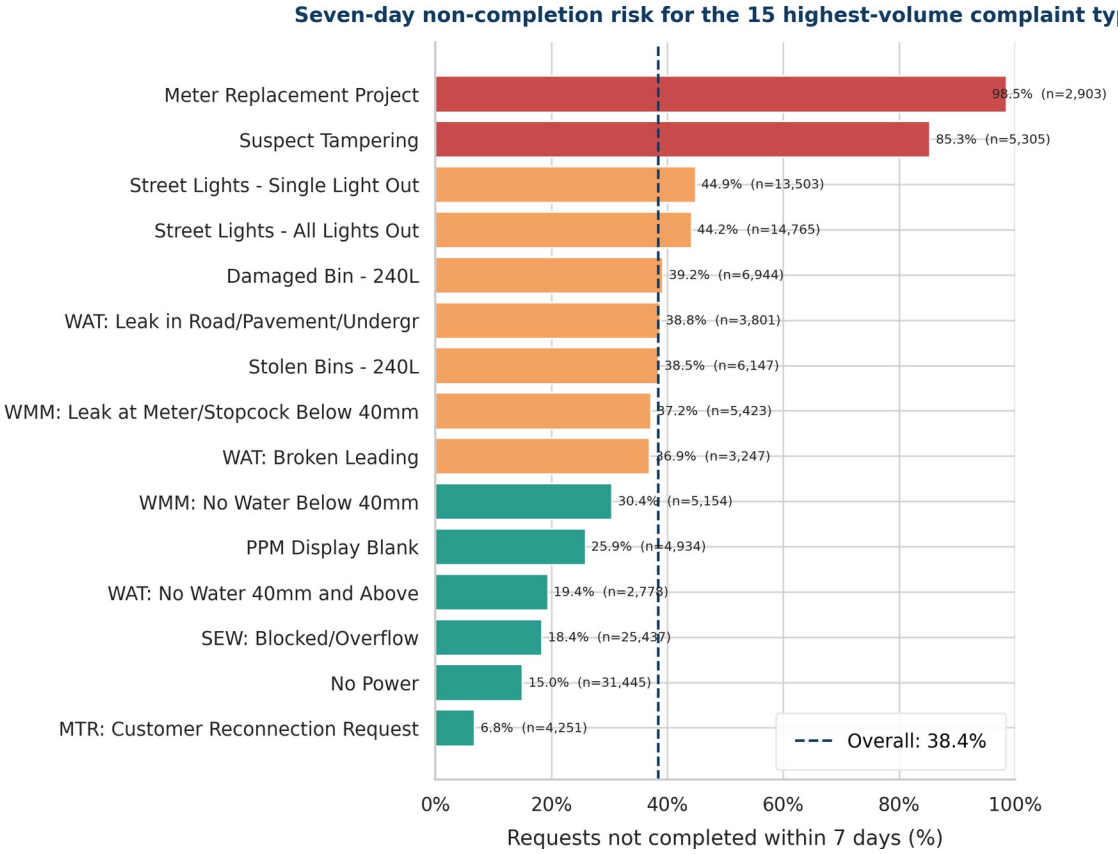


Figure 2.2C. Risk among the highest-volume complaint types.

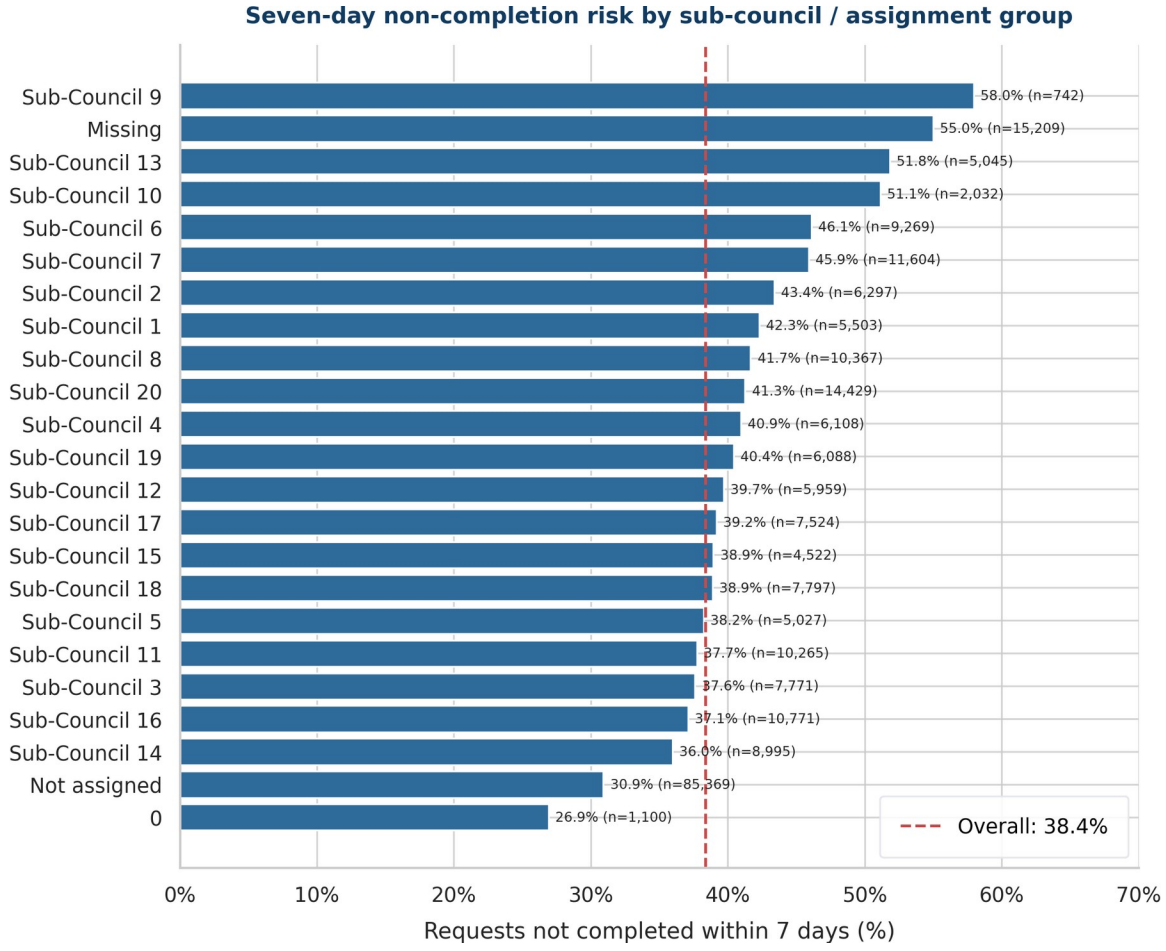


Figure 2.2D. Risk by sub-council/assignment group.

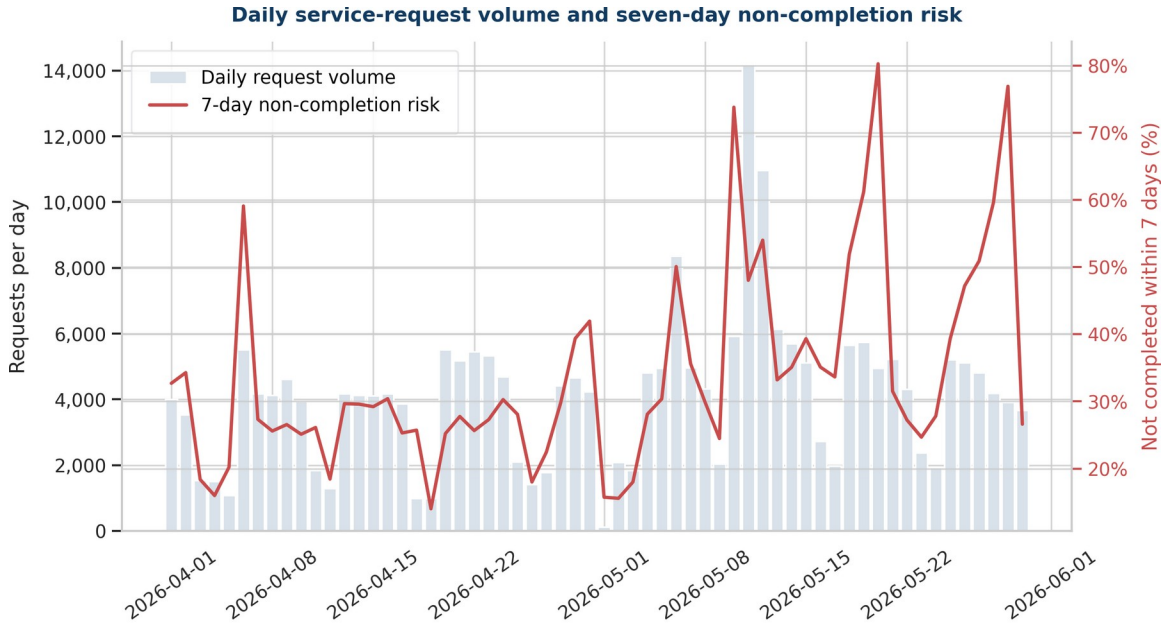


Figure 2.2E. Daily volume and risk.

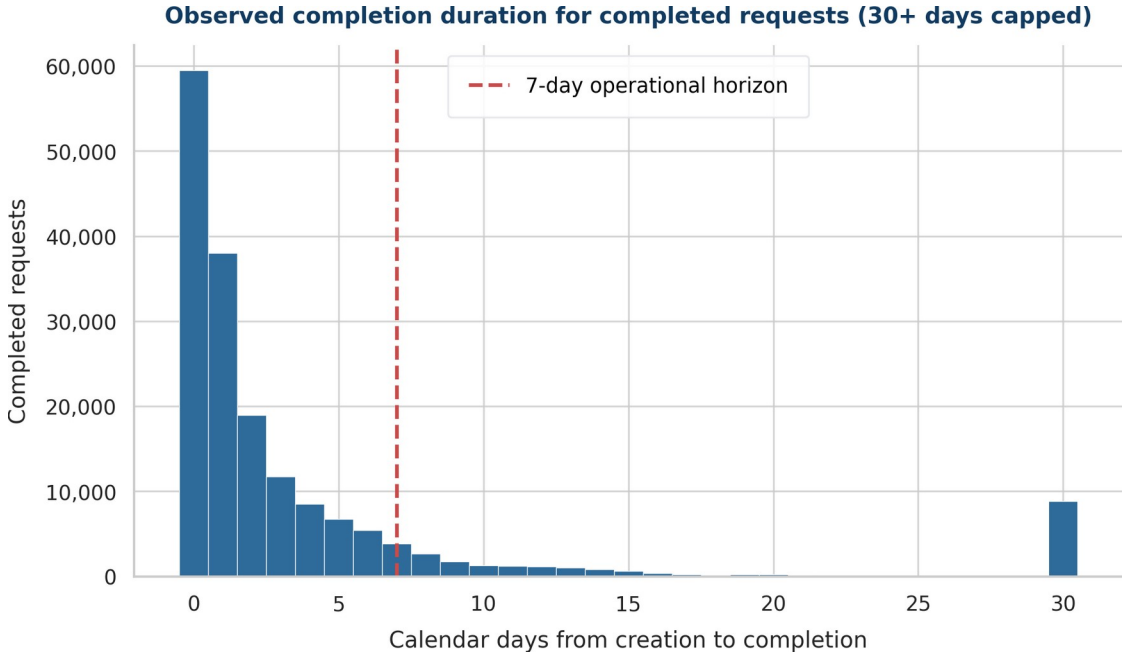


Figure 2.2F. Completion-duration distribution.

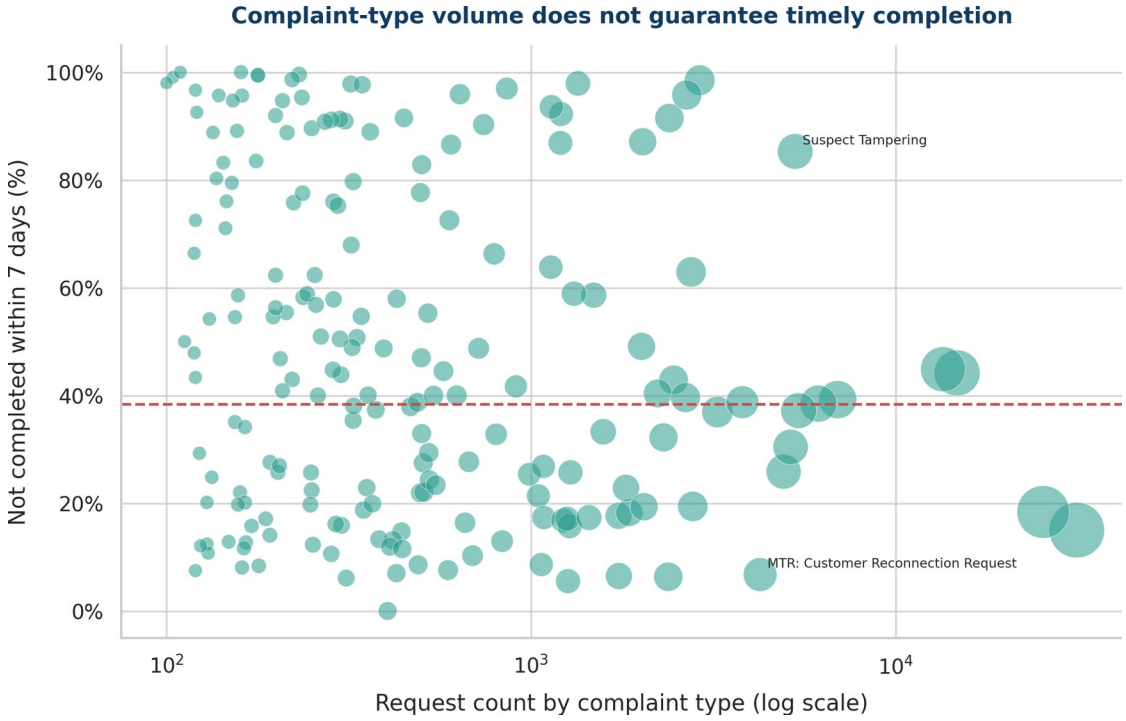


Figure 2.2G. Complaint volume versus risk.

2.3 Chronological split and test protection (3 marks)

WHAT THE QUESTION ASKS

Split before learned transformations. Justify train/validation/test design, including time constraints, and reserve the test for final evaluation only.

Answer to 2.3

SPLIT	CREATION DATES	ROWS	AT-RISK RATE	PERMITTED USE

Training	1 Apr-12 May 2026	169,005	35.3%	Fit transformations/models.
Validation	13-21 May 2026	43,247	45.7%	Select model, epoch, parameters, threshold.
Test	22-30 May 2026	35,543	44.2%	Use once for final evaluation.

We used a time-based split because the model must predict later requests from earlier data. The class rate changes over time, so a random split could hide this drift. All main models used the same rows. The test set was not used to choose features, models, stopping points, hyperparameters or the threshold.

Code used for 2.3

```
# Split before fitting any learned transformation.
local_day = df["Created_On_Date"].dt.tz_convert(
    "Africa/Johannesburg"
).dt.date
train_mask = local_day <= pd.Timestamp("2026-05-12").date()
validation_mask = (
    (local_day >= pd.Timestamp("2026-05-13").date()) &
    (local_day <= pd.Timestamp("2026-05-21").date())
)
test_mask = local_day >= pd.Timestamp("2026-05-22").date()
assert train_mask.sum() == 169005
assert validation_mask.sum() == 43247
assert test_mask.sum() == 35543
```

Question 2.3 - chronological masks created before learning

2.4 Leakage-safe reproducible preparation pipeline (5 marks)

> **WHAT THE QUESTION ASKS**
>
> Implement cleaning, encoding, scaling, imputation, feature engineering and imbalance treatment inside a reproducible pipeline, explain choices and prevent leakage.

Answer to 2.4

STAGE	IMPLEMENTATION
1. Split	Create chronological masks before learned transformations.
2. Clean	Trim categorical text; preserve Ward as text; map blanks to Missing; parse timestamps consistently.
3. Target	Compute elapsed days and derive risk_7d; no completion = at risk.
4. Features	Location/service categories; day-of-week, day index/month/weekend; coordinate-quality indicators.
5. Model-specific encoding	One-hot logistic; training-fitted target/frequency encoding for MLP; integer category codes for LightGBM; learned embeddings.
6. Scaling/imputation	Scale numeric features for linear/neural models; explicit Missing categories rather than target-leaking imputation.
7. Imbalance	Training-partition class weighting where implemented; no synthetic oversampling across time.
8. Lock/test	Fit and select with training/validation, then test once.

To prevent leakage, we excluded Completed_Date and Changed_on because they are known after creation. We also excluded Notification, Notification_type and ObjectId because they act like identifiers, and Notifications_Created because it is almost constant. Encoders, scalers, category maps, weights and early-stopping choices were fitted on training data only. Seed 42 and saved fitted objects support reproducibility.

Code used for 2.4

```
# Leakage-safe feature table.
CAT = ["Sub_Council", "Ward", "Suburb", "C3_Complaint_Type",
       "Work_Center", "day_of_week"]
NUM = ["day_index", "day_of_month", "is_weekend",
       "coord_missing", "coord_zero"]
X = df[CAT + NUM]
y = df["risk_7d"]
# Completed_Date, Changed_on, Notification, Notification_type,
# Notifications_Created and ObjectId are excluded from X.

logistic_preprocessing = ColumnTransformer([
    ("categorical", OneHotEncoder(
        handle_unknown="ignore", min_frequency=10
    ), CAT),
    ("numeric", StandardScaler(), NUM),
])
logistic_pipeline = Pipeline([
    ("preprocessing", logistic_preprocessing),
    ("model", SGDClassifier(
        loss="log_loss", class_weight="balanced", random_state=42
    )),
])
logistic_pipeline.fit(X.loc[train_mask], y[train_mask])
```

Question 2.4 - leakage-safe feature table and example fitted pipeline

KEY POINT

The audit checked class balance, duplicates, missing values, invalid values, outliers and leakage. Seven figures supported the choices. The time-based split and training-only preprocessing make the work reproducible.

INTRODUCTION TO DATA SCIENCE

QUESTION 3 - REVISED ANSWER

Baseline and Model Set

Separate answers for Sub-questions 3.1, 3.2, 3.3, 3.4 and 3.5

DOCUMENT STRUCTURE

Each sub-question is answered under its own number. The answers are not combined. Where a model or training procedure is required, the relevant Python code used in the project is shown directly under that sub-question.

Common experimental setup used by all Question 3 models

All models use the same time-based training, validation and test sets. The setup is shown once to avoid repeating the same code. The test set is created here but is not used to select a model in Question 3.

```
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

CAT = ["Sub_Council", "Ward", "Suburb", "C3_Complaint_Type",
       "Work_Center", "day_of_week"]
NUM = ["day_index", "day_of_month", "is_weekend",
       "coord_missing", "coord_zero"]
FEATURES = CAT + NUM
TARGET = "risk_7d"

# Chronological masks are created before fitting any learned transformation.
local_day = df["created_local"].dt.date
train_mask = local_day <= pd.Timestamp("2026-05-12").date()
validation_mask = (
    (local_day >= pd.Timestamp("2026-05-13").date()) &
    (local_day <= pd.Timestamp("2026-05-21").date())
)
test_mask = local_day >= pd.Timestamp("2026-05-22").date()

X = df[FEATURES]
y = df[TARGET].to_numpy(np.int8)
X_train = X.loc[train_mask].reset_index(drop=True)
X_validation = X.loc[validation_mask].reset_index(drop=True)
X_test = X.loc[test_mask].reset_index(drop=True)
y_train, y_validation, y_test = y[train_mask], y[validation_mask], y[test_mask]
```

Shared split and feature setup used by Questions 3.1-3.5

3.1 Naïve baseline (2 marks)

WHAT THE QUESTION ASKS

Build a transparent naïve baseline, for example a majority or stratified DummyClassifier, and explain what it establishes.

Answer to 3.1

We trained a DummyClassifier using strategy="prior". It learns only the class proportions and predicts the majority class at threshold 0.50. It does not use any complaint, location, work-centre or date features.

The baseline gives the minimum result that the main models should beat. It achieved 55.8% test accuracy, but at-risk recall was 0%, macro-F1 was 0.358 and ROC-AUC was 0.500. It missed all 15,701 at-risk test requests, showing that accuracy alone is misleading.

> **KEY POINT**
> The baseline is transparent and intentionally weak. It proves that a useful solution must identify both classes and improve macro-F1 and ranking performance, not merely predict the majority class.

Code used for 3.1

```
# 3.1 - transparent naive baseline
baseline = DummyClassifier(strategy="prior")
baseline.fit(np.zeros((len(y_train), 1)), y_train)

baseline_probability = np.full(len(y_test), baseline.class_prior_[1])
baseline_prediction = (baseline_probability >= 0.50).astype("int8")
baseline_macro_f1 = f1_score(
    y_test, baseline_prediction, average="macro", zero_division=0
)
```

Question 3.1 - DummyClassifier baseline training and prediction

3.2 Four main models on identical splits (10 marks)

> **WHAT THE QUESTION ASKS**
> Train at least four main models on identical data splits: at least two traditional models and at least two neural-learning models. The naïve baseline does not count among the four.

Answer to 3.2

We trained four main binary classifiers on the same 169,005 training rows and compared them on the same 43,247 validation rows. The 35,543 test rows were kept for the final evaluation in Question 4.

MODEL	FAMILY	VALIDATION MACRO-F1	ROLE
Regularised logistic regression	Traditional	0.587	Transparent linear comparator
LightGBM	Traditional	0.575	Non-linear boosted-tree comparator
Dense MLP	Neural	0.701	Required multilayer perceptron
Entity-embedding network	Neural	0.612	Task-appropriate tabular neural architecture

All models used the same features, target, time split and random seed. Each model used suitable preprocessing, but every learned transformation was fitted on training data only. The DummyClassifier was separate and did not count as one of the four main models.

3.2(a) Code - traditional model 1: regularised logistic regression

```
# 3.2 - traditional model 1
logistic_preprocessing = ColumnTransformer([
    ("categorical", OneHotEncoder(
        handle_unknown="ignore", min_frequency=10, dtype=np.float32
    ), CAT),
    ("numeric", StandardScaler(), NUM),
], sparse_threshold=0.30)

logistic_model = SGDClassifier(
    loss="log_loss", penalty="elasticnet", alpha=1e-5,
```

```
    l1_ratio=0.05, class_weight="balanced", max_iter=60,
    tol=1e-4, random_state=SEED, average=True
)
logistic_pipeline = Pipeline([
    ("preprocessing", logistic_preprocessing),
    ("model", logistic_model),
])
logistic_pipeline.fit(X_train, y_train)
logistic_validation_probability = logistic_pipeline.predict_proba(
    X_validation
)[:, 1]
```

Question 3.2 - logistic-regression training pipeline

3.2(b) Code - traditional model 2: LightGBM

```
# 3.2 - traditional model 2
category_maps = fit_category_maps(X_train)
encoded_train = encode_for_lightgbm(X_train, category_maps)
encoded_validation = encode_for_lightgbm(X_validation, category_maps)

lightgbm_model = LGBMClassifier(
    n_estimators=350, learning_rate=0.05, num_leaves=31,
    min_child_samples=50, subsample=0.90, colsample_bytree=0.90,
    reg_alpha=0.0, reg_lambda=1.0, class_weight="balanced",
    random_state=SEED, n_jobs=-1, verbosity=-1
)
lightgbm_model.fit(
    encoded_train, y_train,
    categorical_feature=list(range(len(CAT))),
    eval_set=[(encoded_validation, y_validation)],
    eval_metric="binary_logloss",
    callbacks=[early_stopping(40, verbose=False), log_evaluation(0)]
)
```

Question 3.2 - LightGBM training with validation early stopping

3.2(c) Code - neural model 1: dense MLP

```
# 3.2 - neural model 1: dense MLP
mlp = MLPClassifier(
    hidden_layer_sizes=(64, 32), activation="relu", solver="adam",
    alpha=1e-4, batch_size=2048, learning_rate_init=0.001,
    max_iter=1, warm_start=True, shuffle=True, random_state=SEED
)
best_mlp, best_score, stale_epochs = None, -1.0, 0
for epoch in range(1, 26):
    if epoch == 1:
        mlp.partial_fit(dense_train, y_train, classes=np.array([0, 1]))
    else:
        mlp.partial_fit(dense_train, y_train)
    probability = mlp.predict_proba(dense_validation)[:, 1]
    score = f1_score(
        y_validation, probability >= 0.50, average="macro"
    )
    if score > best_score + 1e-4:
        best_mlp, best_score = copy.deepcopy(mlp), score
        stale_epochs = 0
    else:
        stale_epochs += 1
        if stale_epochs >= 4:
            break
```

Question 3.2 - dense MLP training and external early stopping

3.2(d) Code - neural model 2: entity-embedding network

```
# 3.2 - neural model 2: entity-embedding network
embedding_maps = fit_category_maps(X_train)
numeric_mean = X_train[NUM].to_numpy(np.float32).mean(axis=0)
numeric_std = X_train[NUM].to_numpy(np.float32).std(axis=0)
numeric_std = np.where(numeric_std < 1e-6, 1, numeric_std)
cat_train, num_train = encode_for_embedding(
    X_train, embedding_maps, numeric_mean, numeric_std
)
cat_validation, num_validation = encode_for_embedding(
    X_validation, embedding_maps, numeric_mean, numeric_std
)
embedding_model, embedding_history = train_embedding_network(
    cat_train, num_train, y_train,
    cat_validation, num_validation, y_validation,
    max_epochs=16, patience=4
)
```

Question 3.2 - entity-embedding preprocessing and training call

KEY POINT

We trained two traditional and two neural models using the same fair process. The dense MLP had the best unoptimised validation macro-F1. The entity-embedding model gave a second neural method for category-heavy tabular data.

3.3 Model-family and neural-architecture compliance (3 marks)

WHAT THE QUESTION ASKS

Traditional examples include Logistic Regression, Decision Tree/Random Forest, SVM, Naïve Bayes or gradient boosting. Neural requirements include an MLP and one task-appropriate second architecture, such as CNN, GRU/LSTM or an approved tabular neural network.

Answer to 3.3

The two traditional models were regularised logistic regression and LightGBM. The two neural models were a dense MLP and an entity-embedding network for tabular data.

A CNN was not suitable because the data is not images. A GRU/LSTM was not suitable because each row is one service request, not a time sequence. The entity-embedding network suits high-cardinality fields such as Suburb, C3_Complaint_Type and Work_Center. It learns a short numeric vector for each category, combines these vectors with scaled numeric features, and sends them through a 64-unit ReLU layer for binary prediction.

APPROVAL CONDITION - EVIDENCE REQUIRED

The entity-embedding network is the implemented second tabular neural architecture. The project template requires lecturer approval for an approved tabular neural network. No lecturer-approval evidence has been supplied, so approval is not claimed. Before submission, add a dated email, LMS message, signed note or other lecturer confirmation, or replace the architecture with one already accepted by the lecturer.

The entity-embedding network is the implemented second neural architecture. Because the template says the second tabular architecture should be approved, lecturer approval should be recorded before final submission.

Code used for 3.3 - second neural architecture

```
class EntityEmbeddingNetwork:
    def __init__(self, cardinalities, numeric_count, hidden_units=64):
        self.embedding_dimensions = [
            min(12, max(3, round(math.sqrt(max(2, n)))))
            for n in cardinalities
        ]
        self.embeddings = [
            np.random.normal(0, 0.03, (n + 1, d)).astype(np.float32)
            for n, d in zip(cardinalities, self.embedding_dimensions)
        ]
        input_width = sum(self.embedding_dimensions) + numeric_count
        self.weight_1 = he_initialise(input_width, hidden_units)
        self.bias_1 = np.zeros(hidden_units, np.float32)
        self.weight_2 = he_initialise(hidden_units, 1)
        self.bias_2 = np.zeros(1, np.float32)

    def forward(self, categorical, numeric):
        pieces = [embedding[categorical[:, index]]
                  for index, embedding in enumerate(self.embeddings)]
        inputs = np.concatenate(pieces + [numeric], axis=1)
        hidden = np.maximum(inputs @ self.weight_1 + self.bias_1, 0)
        logits = (hidden @ self.weight_2 + self.bias_2).ravel()
        return logits, hidden
```

Question 3.3 - entity-embedding network architecture

3.4 Architecture, hyperparameters and reproducible training procedure (5 marks)

WHAT THE QUESTION ASKS

Document architecture/hyperparameters, loss, optimiser, epochs, batch size, regularisation, early stopping, seeds, compute environment and reproducible training procedure.

Answer to 3.4

MODEL	ARCHITECTURE / HYPERPARAMETERS	LOSS / OPTIMISER	REGULARISATION / STOPPING
Logistic regression	One-hot categories + scaled numeric features; SGD; max 60 passes	Log loss; SGD	Elastic-net alpha 1e-5; l1_ratio 0.05; balanced class weights
LightGBM	350-tree ceiling; learning rate 0.05; 31 leaves; min child 50; subsample and column sample 0.90	Binary log loss; gradient boosting	L2=1; balanced weights; early stopping patience 40
Dense MLP	162 inputs; hidden 64-32; ReLU; batch 2,048; max 25 epochs	Binary log loss; Adam at 0.001	L2 alpha 1e-4; best validation epoch retained; patience 4; selected epoch 5
Entity embeddings	Six embedding tables (3-12 dimensions); numeric inputs; hidden 64 ReLU; batch 4,096; max 16 epochs	Weighted binary cross-entropy; Adam at 0.0015	Weight decay 1e-5; class weighting; patience 4; selected epoch 2

The experiment used Python 3.12 on CPU with NumPy, pandas, scikit-learn 1.5.1 and LightGBM 4.6.0. Seed 42 was fixed for Python, NumPy and every model. The split dates, features and target were fixed. All models used the same training and validation rows. Fitted preprocessing, models and neural training histories were saved in a joblib file.

Training steps: (1) load and clean the CSV; (2) create the seven-day target; (3) create the time-based split; (4) fit preprocessing on training data only; (5) train all models with seed 42; (6) choose epochs or stopping points using validation only; (7) compare validation macro-F1; and (8) save the fitted objects. The test set was not used to choose a model.

Code used for 3.4 - common comparison and saved reproducibility bundle

```
# Common validation comparison: identical validation cases and metric.
validation_probabilities = {
    "Dummy baseline": np.full(len(y_validation), baseline.class_prior_[1]),
    "Regularized logistic": logistic_pipeline.predict_proba(X_validation)[:, 1],
    "LightGBM": lightgbm_model.predict_proba(encoded_validation)[:, 1],
    "Dense MLP": best_mlp.predict_proba(dense_validation)[:, 1],
    "Entity-embedding network": embedding_model.predict_proba(
        cat_validation, num_validation
    ),
}
for name, probability in validation_probabilities.items():
    prediction = (probability >= 0.50).astype("int8")
    print(name, f1_score(
        y_validation, prediction, average="macro", zero_division=0
    ))

# Persist fitted models, preprocessing and neural histories.
model_bundle = {
    "seed": SEED, "features": FEATURES, "baseline": baseline,
    "logistic": logistic_pipeline, "lightgbm": lightgbm_model,
    "category_maps": category_maps, "mlp": best_mlp, "mlp_history": mlp_history,
    "embedding_model": embedding_model, "embedding_history": embedding_history,
}
joblib.dump(model_bundle, "question3_model_bundle.joblib", compress=3)
```

Question 3.4 - identical validation comparison and model persistence

3.5 Rationale for each model family (2 marks)

WHAT THE QUESTION ASKS

Provide a concise rationale linking each model family to the data and problem.

Answer to 3.5

MODEL FAMILY	WHY IT FITS THIS DATA AND PROBLEM
Regularised logistic regression	Provides a transparent linear benchmark. It tests whether the request outcome can be explained by additive relationships after encoding categories. Coefficients can be inspected, but the model may miss complex interactions.
LightGBM gradient boosting	Municipal request data is structured tabular data with non-linear interactions among complaint type, work centre, time and location. Boosted trees handle such interactions efficiently and require little numeric scaling.
Dense MLP	Tests whether distributed non-linear combinations of encoded tabular features improve prediction beyond linear and tree baselines. It satisfies the required MLP architecture and supports learning curves and epoch-based early stopping.
Entity-embedding neural network	High-cardinality categories can be represented as learned vectors rather than extremely wide one-hot matrices. The network can learn similarity among suburbs, complaints and work centres while combining these with numeric date/quality features.

The four model families test different ideas: linear effects, tree rules and interactions, dense neural combinations, and learned category vectors. This gives a fair comparison instead of using four similar algorithms.

KEY POINT

The model families suit a large mixed-type municipal dataset. No model is assumed to be best; validation results decide.

INTRODUCTION TO DATA SCIENCE

QUESTION 4 - REVISED ANSWER

Evaluation, Comparison and Overfitting

Separate answers for Sub-questions 4.1, 4.2, 4.3 and 4.4

DOCUMENT STRUCTURE

Each sub-question is answered under its exact number. Answers are not combined. The applicable code is shown directly under the metric, report, plot, comparison or diagnostic that it produced.

Evaluation protocol shared by Questions 4.1-4.4

The four main models used the same time-based training, validation and locked test rows. The positive class is “not completed within seven days”. The validation set selected the best model; the test set was not used for selection. A separate three-fold expanding-window analysis measured changes over time without using the locked test set.

SPLIT	DATES	ROWS	ROLE
Training	1 Apr-12 May 2026	169,005	Fit preprocessing and models
Main validation	13-21 May 2026	43,247	Model selection and early stopping
Locked test	22-30 May 2026	35,543	Identical test evaluation cases

4.1 Common test metrics for every main model (4 marks)

WHAT THE QUESTION ASKS

For every main model, report the same test metrics. Include accuracy and macro-F1; add precision, recall, weighted-F1 and ROC-AUC as appropriate. For imbalance, include PR-AUC or justify its omission.

Answer to 4.1

We evaluated logistic regression, LightGBM, the dense MLP and the entity-embedding network on the same 35,543 locked test cases. The base-model comparison used threshold 0.50. We report accuracy, macro-F1, weighted-F1, at-risk precision and recall, ROC-AUC and PR-AUC.

MODEL	ACC.	MACRO-F1	W-F1	RISK PREC.	RISK REC.	ROC-AUC	PR-AUC
Regularized logistic	0.601	0.584	0.575	0.528	0.906	0.760	0.731
LightGBM	0.697	0.697	0.696	0.620	0.810	0.791	0.788
Dense MLP	0.642	0.639	0.636	0.565	0.826	0.758	0.732
Entity-embedding network	0.643	0.632	0.625	0.558	0.920	0.814	0.786

LightGBM had the best base-model test accuracy (0.697) and macro-F1 (0.697). The entity-embedding model had the best ROC-AUC (0.814) and at-risk recall (0.920), but its precision was only 0.558, so it produced many false positives. We included PR-AUC because the at-risk class is smaller and the precision-recall trade-off matters for the review queue.

Code used for 4.1

```
def evaluate_binary_model(y_true, probability, threshold=0.50):
    prediction = (np.asarray(probability) >= threshold).astype("int8")
    return {
        "accuracy": accuracy_score(y_true, prediction),
        "macro_f1": f1_score(y_true, prediction, average="macro"),
        "precision_risk": precision_score(y_true, prediction),
        "recall_risk": recall_score(y_true, prediction),
        "weighted_f1": f1_score(y_true, prediction, average="weighted"),
        "roc_auc": roc_auc_score(y_true, probability),
        "pr_auc": average_precision_score(y_true, probability),
    }

for model_name, probability in test_probabilities.items():
    test_metrics[model_name] = evaluate_binary_model(
        y_test, probability, threshold=0.50
    )
```

Question 4.1 - common metric calculation for every model

4.2 Classification reports, confusion matrices and ROC curves (5 marks)

WHAT THE QUESTION ASKS

Provide a classification report and confusion matrix for every model; provide ROC curves on one common figure. For multiclass, use one-vs-rest macro/micro curves. This project is binary, so one ROC curve per model is sufficient.

Answer to 4.2

Each main model has a classification report with precision, recall, F1 and support for both classes. All confusion matrices use class 0 for completed within seven days and class 1 for at risk. ROC curves use predicted probabilities and are plotted together for comparison.

MODEL	CLASS	PREC.	RECALL	F1	SUPPORT
Regularized logistic	Within 7d	0.829	0.359	0.501	19,842
Regularized logistic	At risk	0.528	0.906	0.667	15,701
LightGBM	Within 7d	0.802	0.608	0.691	19,842
LightGBM	At risk	0.620	0.810	0.703	15,701
Dense MLP	Within 7d	0.783	0.497	0.608	19,842
Dense MLP	At risk	0.565	0.826	0.671	15,701
Entity-embedding network	Within 7d	0.870	0.423	0.570	19,842
Entity-embedding network	At risk	0.558	0.920	0.695	15,701

Code used to generate classification reports and confusion matrices

```
for model_name, probability in test_probabilities.items():
    prediction = (probability >= 0.50).astype("int8")
    reports[model_name] = classification_report(
        y_test, prediction,
        target_names=["Completed within 7 days", "Not completed within 7 days"],
        output_dict=True, zero_division=0
    )
```

```
    matrices[model_name] = confusion_matrix(
        y_test, prediction, labels=[0, 1]
    )

fig, axes = plt.subplots(2, 2, figsize=(10, 8))
for axis, model_name in zip(axes.ravel(), substantive_models):
    sns.heatmap(matrices[model_name], annot=True, fmt=",d",
                cmap="Blues", cbar=False, ax=axis)
    axis.set_title(model_name)
    axis.set_xlabel("Predicted class")
    axis.set_ylabel("Actual class")
```

Question 4.2 - classification reports and confusion-matrix plotting

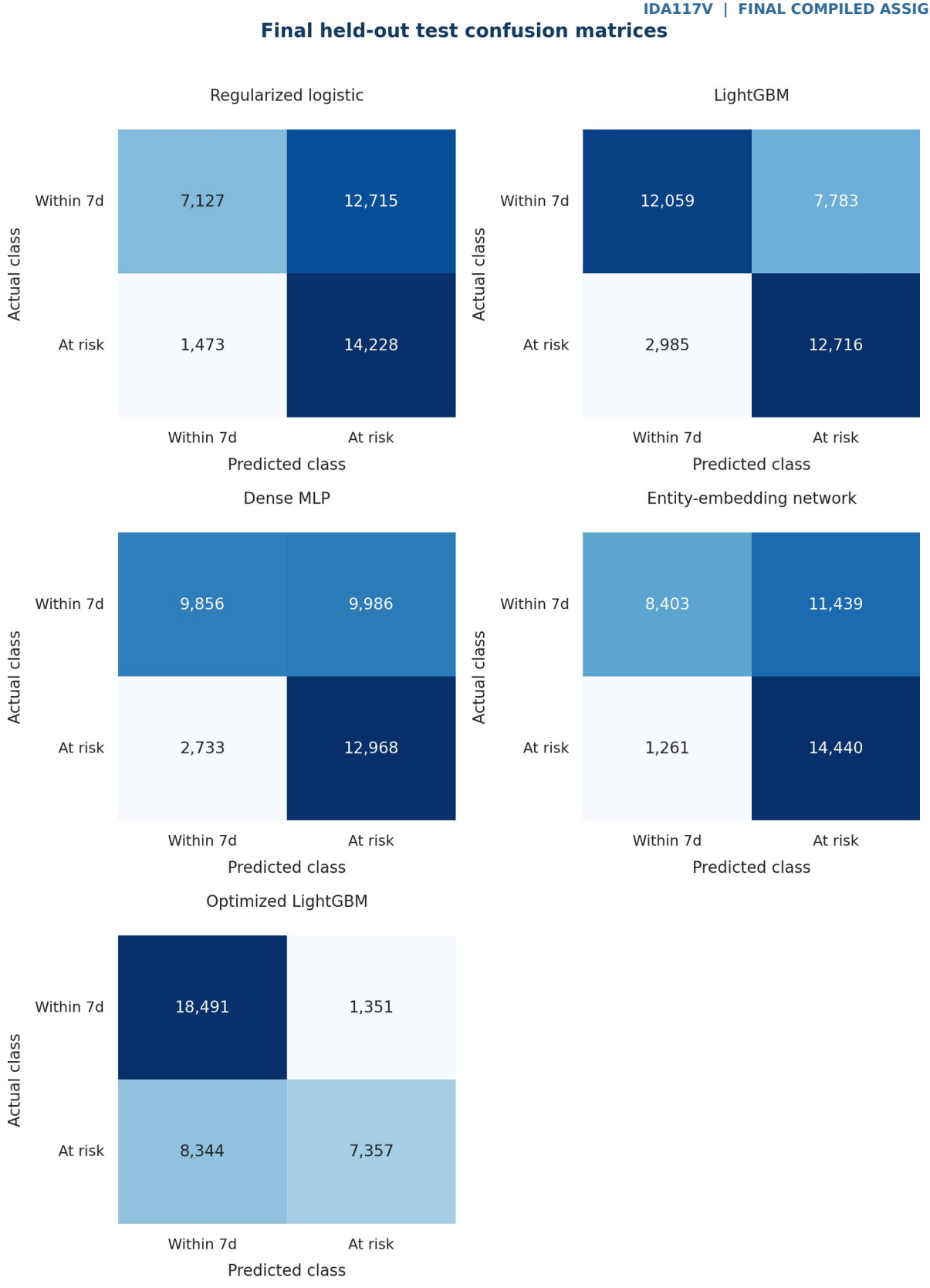


Figure 4.2A. Confusion matrices on the identical locked test cases.

Code used to plot the common ROC figure

```
fig, axis = plt.subplots(figsize=(8, 6))
for model_name in substantive_models:
    probability = test_probabilities[model_name]
```

```
    false_positive_rate, true_positive_rate, _ = roc_curve(
        y_test, probability
    )
    auc = roc_auc_score(y_test, probability)
    axis.plot(false_positive_rate, true_positive_rate,
              label=f"{model_name} (AUC={auc:.3f})")
axis.plot([0, 1], [0, 1], "--", color="grey")
axis.set_xlabel("False-positive rate")
axis.set_ylabel("True-positive rate")
axis.set_title("ROC curves on identical locked test cases")
axis.legend()
plt.tight_layout()
```

Question 4.2 - common ROC-curve plotting code

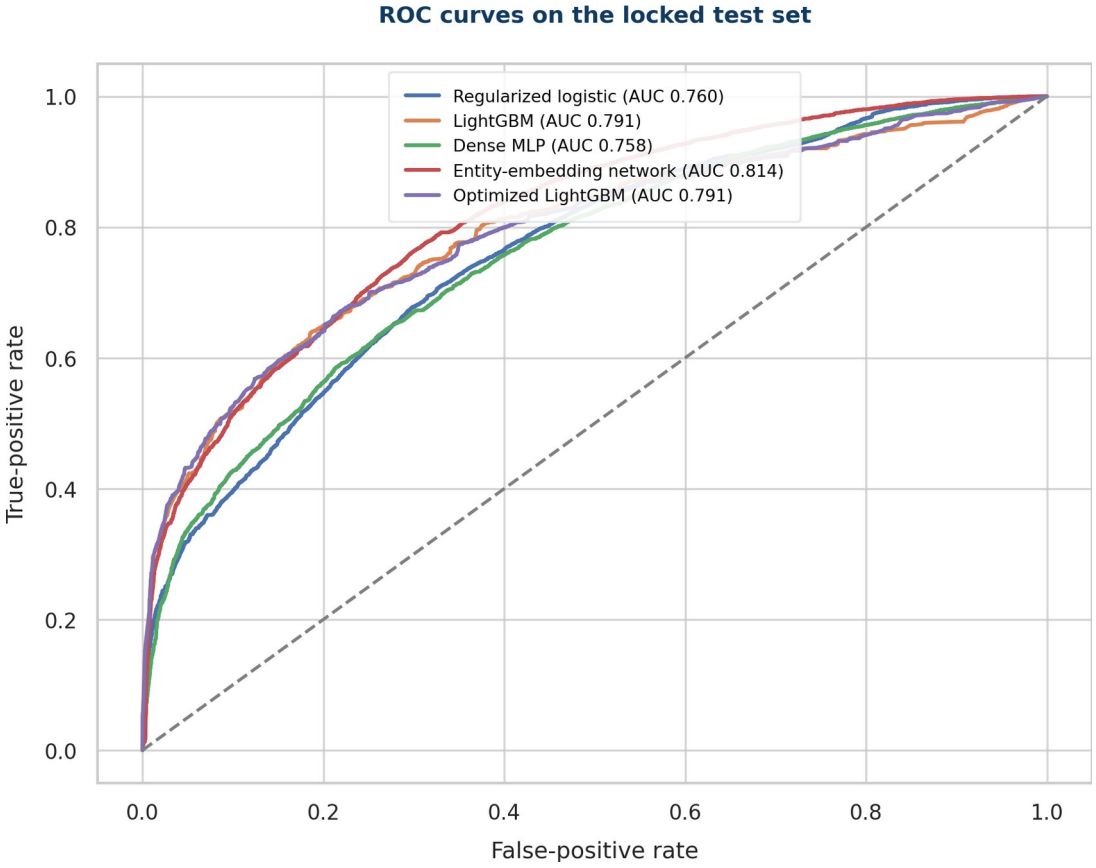


Figure 4.2B. Common ROC curves for the substantive and final reported models.

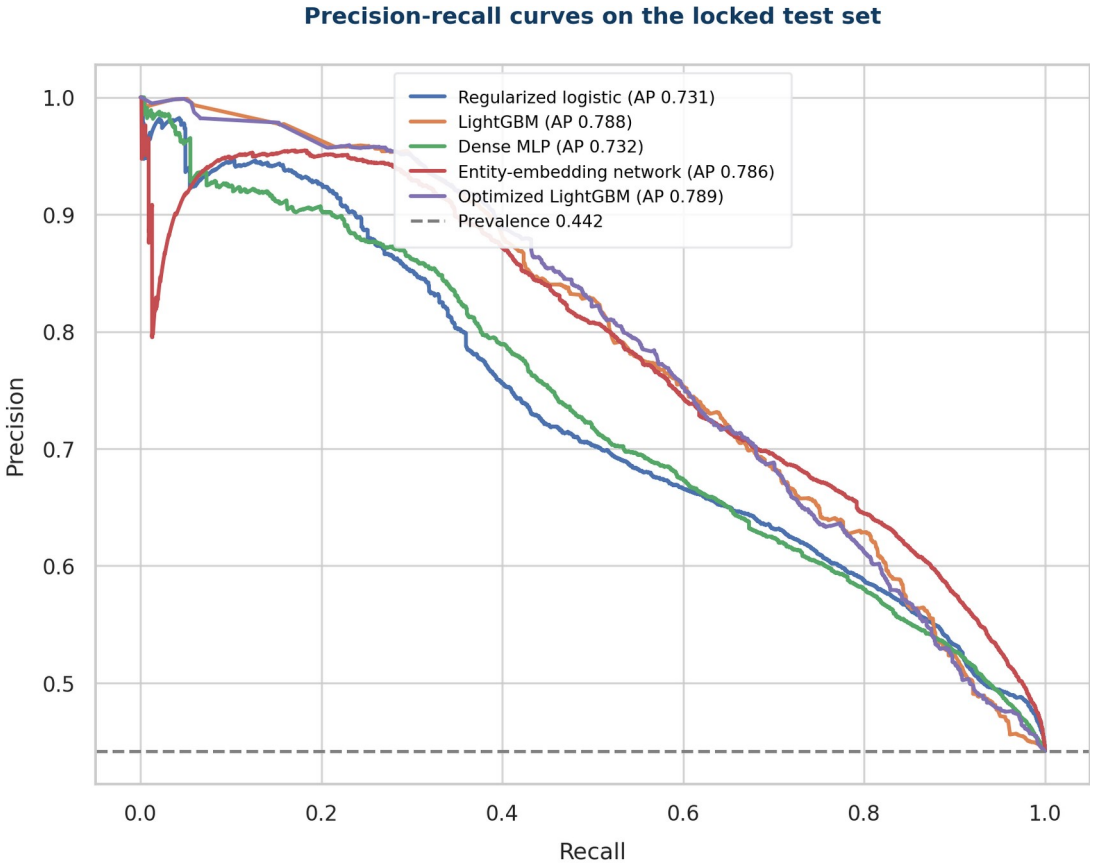


Figure 4.2C. Precision-recall curves included because the at-risk class is important for planning.

KEY POINT

No single score tells the full story. LightGBM was the strongest balanced base model. The embedding model found more at-risk cases but produced more false positives.

4.3 Model comparison, temporal variability and best validation model (3 marks)

WHAT THE QUESTION ASKS

Compare all models with a clear table and bar chart. Show cross-validation means and variability or confidence intervals where feasible. Select the best validation model using validation—not test—performance.

Answer to 4.3

The main validation period was 13-21 May. The dense MLP had the best unoptimised validation macro-F1, 0.701, so it was the best validation model in Question 4. Test results were not used for this decision.

MODEL	VAL ACC.	VAL MACRO-F1	VAL ROC-AUC	VAL PR-AUC
Regularized logistic	0.609	0.587	0.788	0.764
LightGBM	0.591	0.575	0.764	0.774
Dense MLP	0.701	0.701	0.776	0.736
Entity-embedding network	0.628	0.612	0.806	0.790

Code used for the main validation comparison

```
# Main validation comparison: model selection uses validation, not test.
comparison = pd.DataFrame([
    {"model": name,
     "validation_macro_f1": results[name]["validation"]["macro_f1"]}
    for name in substantive_models
]).sort_values("validation_macro_f1", ascending=False)

axis = comparison.plot.bar(
    x="model", y="validation_macro_f1", legend=False,
    color=["#F4A261", "#67597A", "#2A9D8F", "#2F6B9A"]
)
axis.set_ylim(0, 1)
axis.set_ylabel("Validation macro-F1")
plt.tight_layout()
provisional_champion = comparison.iloc[0]["model"]
```

Question 4.3 - validation-led comparison and provisional selection

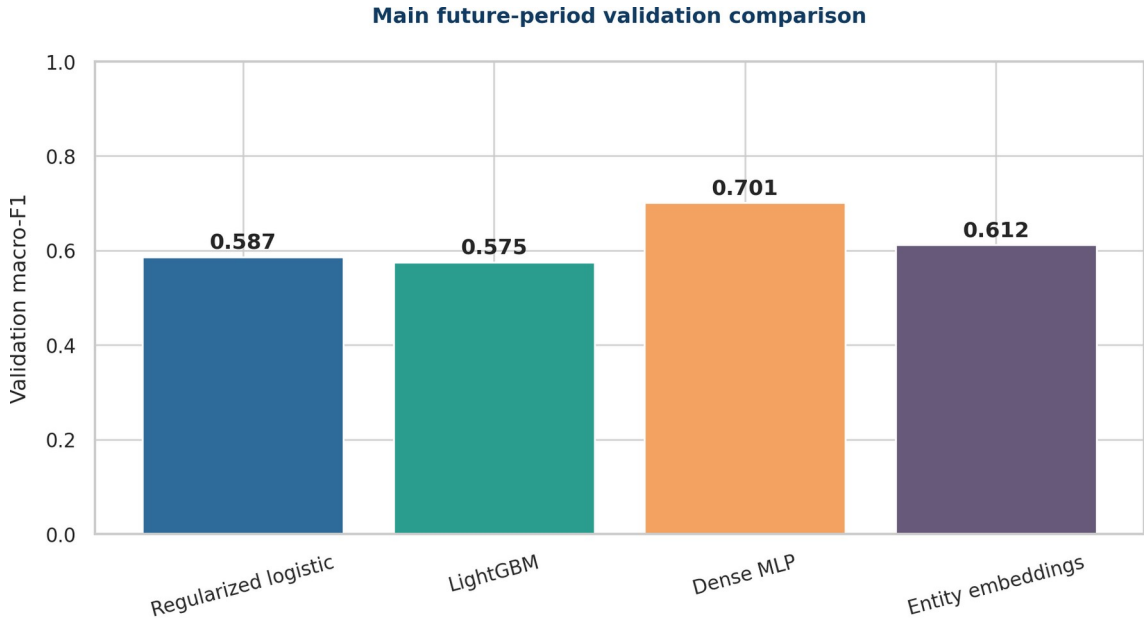


Figure 4.3A. Main validation macro-F1 comparison used for provisional selection.

Three-fold expanding-window variability analysis

We used three expanding-window folds to check performance over time. Each validation window came after its training window and ended before the locked test period. The table shows the mean macro-F1 and sample standard deviation across the three folds.

MODEL	MACRO-F1 MEAN ± SD	ROC-AUC MEAN ± SD	PR-AUC MEAN ± SD
Regularized logistic	0.673 ± 0.057	0.772 ± 0.085	0.702 ± 0.117
LightGBM	0.756 ± 0.038	0.851 ± 0.053	0.811 ± 0.004
Dense MLP	0.728 ± 0.039	0.824 ± 0.051	0.783 ± 0.019
Entity-embedding network	0.745 ± 0.043	0.835 ± 0.056	0.792 ± 0.025

LightGBM had the best temporal-fold mean macro-F1 (0.756) and the lowest standard deviation (0.038). The dense MLP standard deviation was 0.039. This does not replace the main validation decision. It shows that LightGBM was the most stable model across the extra time windows and was a good model to optimise later.

Code used for temporal cross-validation variability

```
# Three expanding-window folds; every validation period follows training.
folds = [
    ("2026-04-01", "2026-04-21", "2026-04-22", "2026-04-30"),
    ("2026-04-01", "2026-04-30", "2026-05-01", "2026-05-09"),
    ("2026-04-01", "2026-05-09", "2026-05-10", "2026-05-21"),
]
fold_rows = []
for train_start, train_end, validation_start, validation_end in folds:
    X_train, y_train, X_validation, y_validation = make_time_fold(
        df, train_start, train_end, validation_start, validation_end
    )
    for model_name in substantive_models:
        fitted_model, fitted_preprocessing = train_fold_model(
            model_name, X_train, y_train, X_validation, y_validation
        )
        probability = predict_fold(
            fitted_model, fitted_preprocessing, X_validation
        )
        fold_rows.append({
            "model": model_name,
            "macro_f1": f1_score(
                y_validation, probability >= 0.50, average="macro"
            )
        })
fold_results = pd.DataFrame(fold_rows)
cv_summary = fold_results.groupby("model")["macro_f1"].agg(
    ["mean", "std"]
)
```

Question 4.3 - expanding-window temporal validation and mean/SD calculation

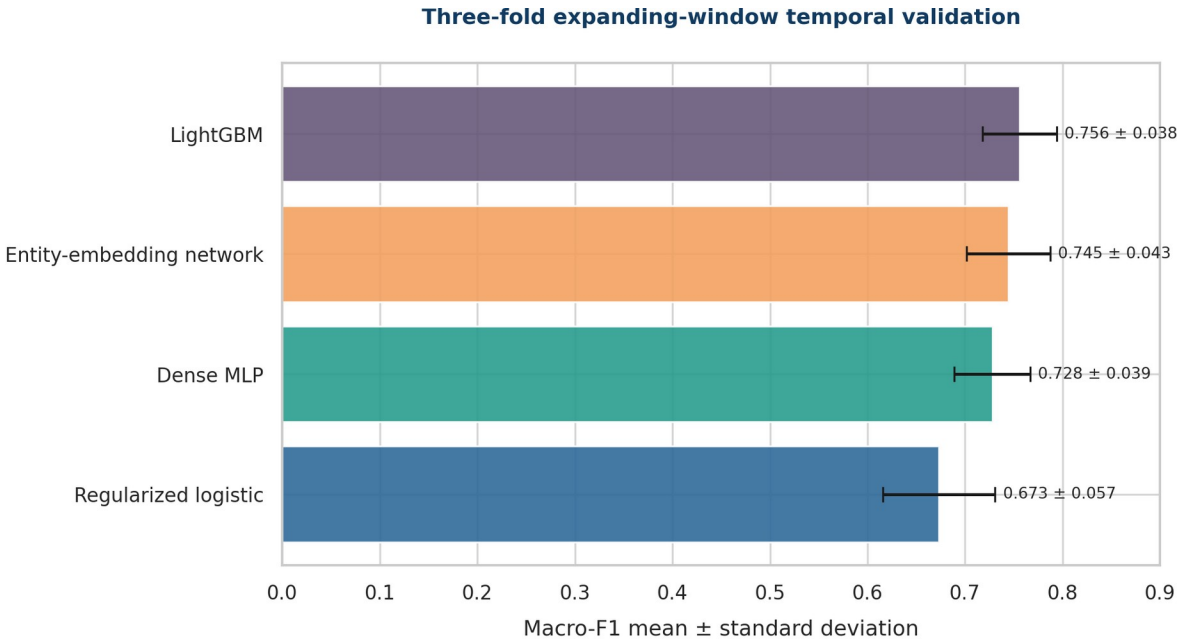


Figure 4.3B. Expanding-window macro-F1 mean and standard deviation.

KEY POINT

Dense MLP was best on the main validation set. LightGBM had the best average and stability across the extra time folds. Both results are useful because they answer different questions.

4.4 Overfitting tests, learning curves and neural curves (4 marks)

WHAT THE QUESTION ASKS

Test for overfitting using train-versus-validation performance and learning curves. For neural models, also show training/validation loss and metric curves. Diagnose gaps and discuss early stopping, dropout, weight decay, pruning or other controls used.

Answer to 4.4

MODEL	TRAIN M-F1	VALIDATION M-F1	GAP	DIAGNOSIS
Regularized logistic	0.706	0.587	0.119	Moderate
LightGBM	0.821	0.575	0.245	High
Dense MLP	0.804	0.701	0.103	Moderate
Entity-embedding network	0.804	0.612	0.193	High

Base LightGBM and the entity-embedding network had the largest train-validation gaps: 0.245 and 0.193. Logistic regression had a gap of 0.119 and the dense MLP 0.103. These gaps show both overfitting and time-based change because the at-risk rate was 35.3% in training and 45.7% in validation.

Code used for train-versus-validation gaps and learning curve

```
gap_rows = []
for model_name in substantive_models:
    train_score = results[model_name]["train"]["macro_f1"]
    validation_score = results[model_name]["validation"]["macro_f1"]
    gap_rows.append({
        "model": model_name,
        "train_macro_f1": train_score,
        "validation_macro_f1": validation_score,
        "gap": train_score - validation_score,
    })
gap_table = pd.DataFrame(gap_rows)

# Data-size learning curve on the fixed future validation period.
for fraction in [0.2, 0.4, 0.6, 0.8, 1.0]:
    n = int(len(y_train) * fraction)
    model = train_base_lightgbm(X_train.iloc[:n], y_train[:n])
    learning_curve_rows.append({
        "train_rows": n,
        "train_macro_f1": macro_f1(y_train[:n], model, X_train.iloc[:n]),
        "validation_macro_f1": macro_f1(y_validation, model, X_validation),
    })
```

Question 4.4 - performance gaps and data-size learning-curve calculation

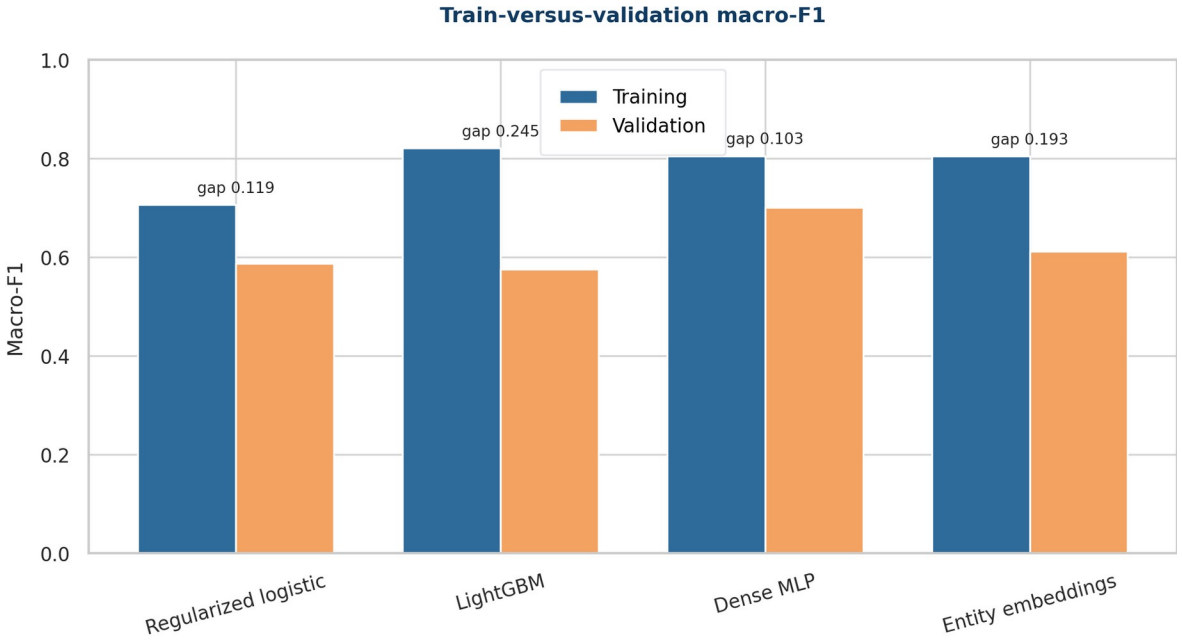


Figure 4.4A. Train-versus-validation macro-F1 gaps.

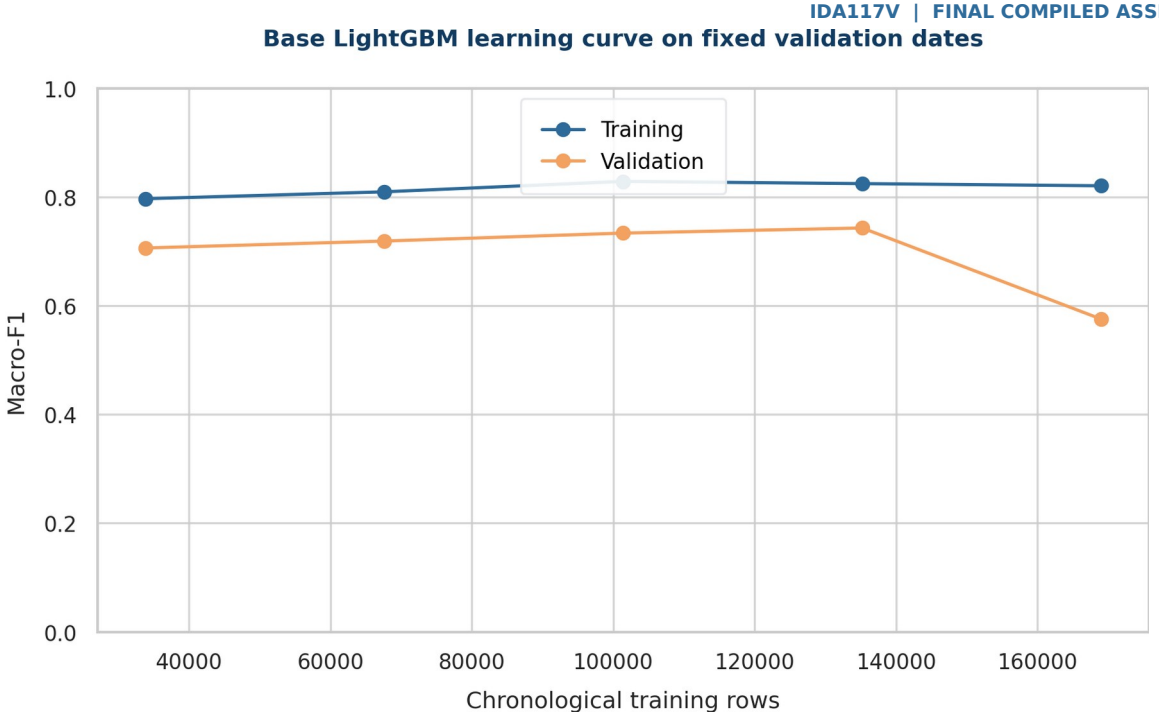


Figure 4.4B. Base LightGBM learning curve on fixed future validation dates.

Validation performance improved as the training window grew from 33,801 to 135,204 rows. It then fell from 0.743 to 0.575 when all 169,005 training rows were used. This does not mean that more data is always harmful. It shows that the latest training period differed from the validation period. Early stopping selected only 14 boosting rounds for the full window.

Neural training and validation curves

The dense MLP reached its best validation macro-F1 at epoch 5 (0.701). After that, training improved but validation loss increased, so epoch 5 was kept. The entity-embedding model reached its best validation macro-F1 at epoch 2 (0.612) and then became worse on validation, so epoch 2 was kept.

Code used to store and plot neural loss/metric curves

```
# During neural training, store both loss and macro-F1 every epoch.
history.append({
    "epoch": epoch,
    "train_loss": log_loss(y_train, train_probability),
    "validation_loss": log_loss(y_validation, validation_probability),
    "train_macro_f1": f1_score(
        y_train, train_probability >= 0.50, average="macro"
    ),
    "validation_macro_f1": f1_score(
        y_validation, validation_probability >= 0.50, average="macro"
    ),
})

# Plot MLP and entity-embedding histories on common axes.
fig, axes = plt.subplots(2, 2, figsize=(10, 7))
for column, (model_name, model_history) in enumerate(histories.items()):
    h = pd.DataFrame(model_history)
    axes[0, column].plot(h.epoch, h.train_loss, label="Train")
    axes[0, column].plot(h.epoch, h.validation_loss, label="Validation")
    axes[1, column].plot(h.epoch, h.train_macro_f1, label="Train")
    axes[1, column].plot(h.epoch, h.validation_macro_f1,
                         label="Validation")
    axes[0, column].legend(); axes[1, column].legend()
plt.tight_layout()
```

Question 4.4 - neural epoch-history collection and plotting

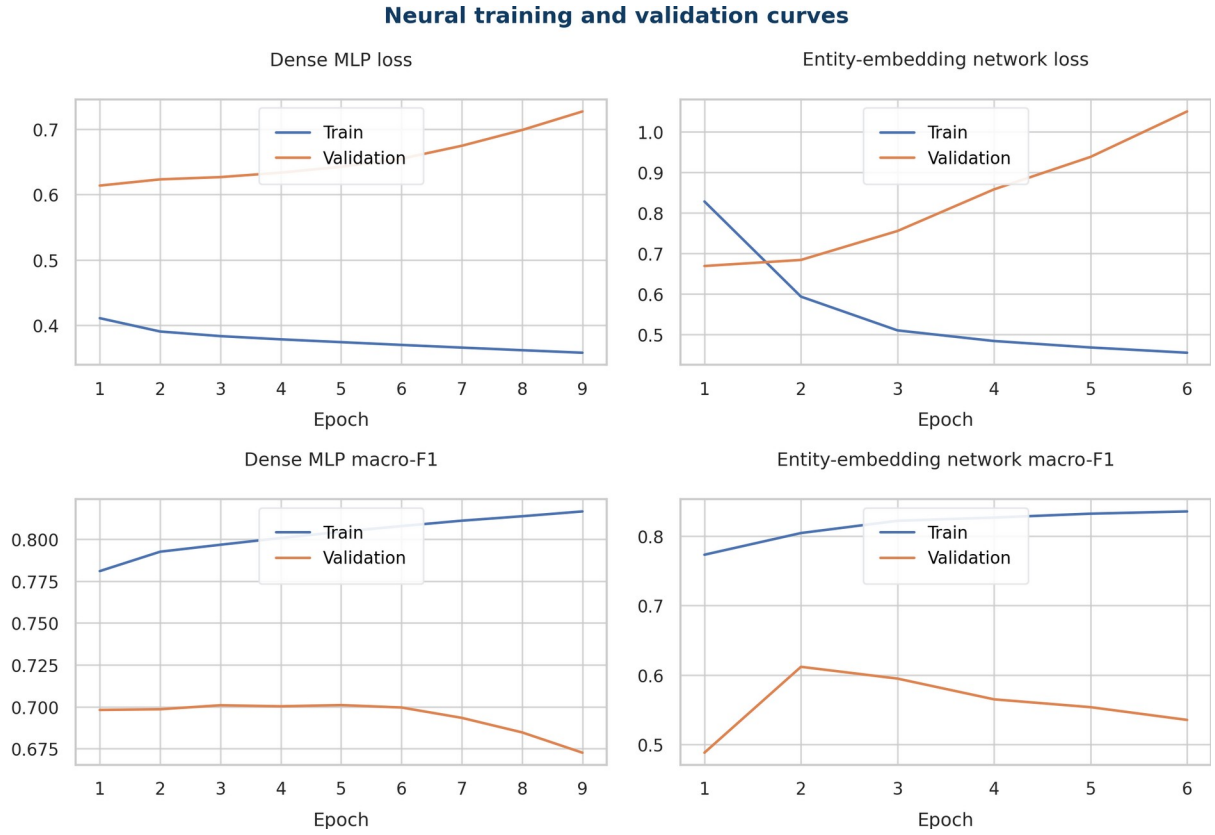


Figure 4.4C. Dense MLP and entity-embedding training/validation loss and macro-F1 curves.

Controls and final diagnosis

- LightGBM: constrained leaves/minimum child size, row and column subsampling, L2 regularisation and validation early stopping.
- Dense MLP: L2 regularisation, small 64-32 architecture, fixed batch size and external patience-based early stopping.
- Entity embeddings: compact 3-12 dimensional embeddings, a single 64-unit hidden layer, class-weighted loss, weight decay and patience-based early stopping.
- Dropout and pruning were not used in this draft. Dropout/category dropout is a justified future neural experiment; stronger tree pruning can be tested during optimization.
- The remaining gap and fold variability mean the live system requires rolling temporal validation, drift monitoring and retraining triggers.

KEY POINT

The models learn useful patterns, but performance changes over time. Early stopping and regularisation reduce overfitting, but drift remains. Any deployment must be monitored.

INTRODUCTION TO DATA SCIENCE

QUESTION 5 - REVISED ANSWER

Champion Optimisation

Separate answers for Sub-questions 5.1, 5.2 and 5.3

DOCUMENT STRUCTURE

Each sub-question is answered independently under its exact number. Applicable code is placed directly under the step it supports. Any item not yet completed is labelled as a placeholder rather than presented as finished.

5.1 Optimise only the strongest validation model (4 marks)

WHAT THE QUESTION ASKS

Optimise one validation-selected candidate using a documented search method. State the search space and scoring rule. Do not optimise every model.

Answer to 5.1

We chose LightGBM for optimisation because it was the strongest traditional model and the most stable across the extra time folds. We tested six fixed settings for tree size, minimum child size, learning rate and L1/L2 regularisation. Each trial allowed up to 900 trees and stopped early after 60 rounds without improvement. We ranked the trials by validation macro-F1 at threshold 0.50. After choosing the best trial, we searched thresholds from 0.20 to 0.80 in steps of 0.01.

TRIAL	LEAVES	MIN CHILD	LR	L1	L2	ITER.	VAL M-F1
1	31	30	0.05	0.0	1.0	14	0.575
2	63	30	0.04	0.0	1.0	23	0.592
3	63	80	0.04	0.1	2.0	20	0.597
4	95	80	0.03	0.2	3.0	33	0.596
5	47	120	0.05	0.1	2.0	15	0.600
6	31	150	0.03	0.3	4.0	33	0.585

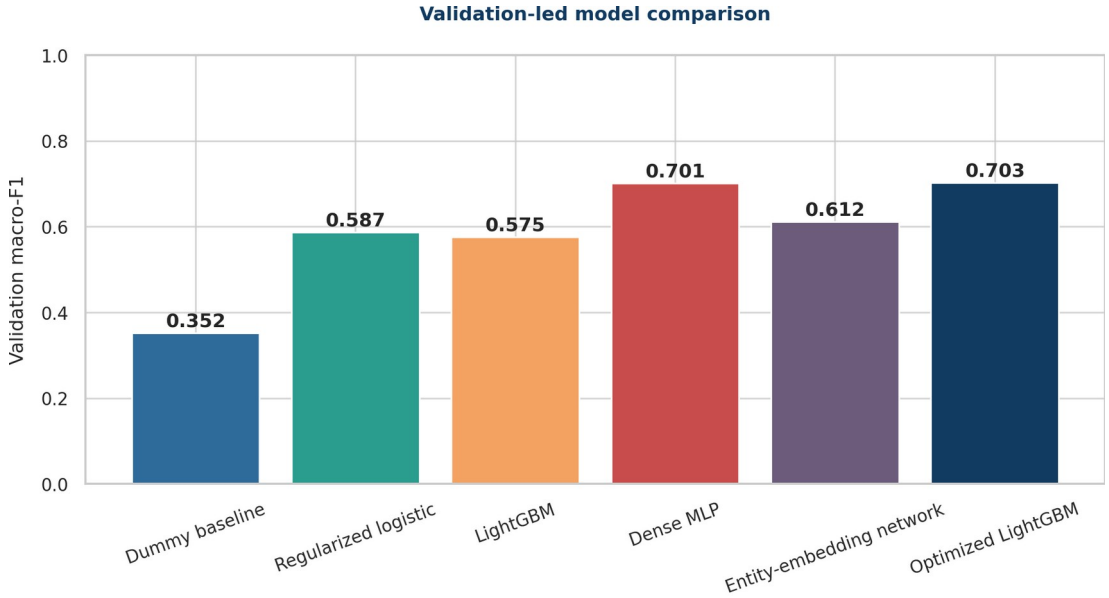


Figure 5.1. Validation-led model comparison before champion optimisation.

Code used for 5.1

```
search_space = [
 {"num_leaves":31,"min_child_samples":30,"reg_lambda":1.0,
  "reg_alpha":0.0,"learning_rate":0.05},
 {"num_leaves":63,"min_child_samples":30,"reg_lambda":1.0,
  "reg_alpha":0.0,"learning_rate":0.04},
 {"num_leaves":63,"min_child_samples":80,"reg_lambda":2.0,
  "reg_alpha":0.1,"learning_rate":0.04},
 {"num_leaves":95,"min_child_samples":80,"reg_lambda":3.0,
  "reg_alpha":0.2,"learning_rate":0.03},
 {"num_leaves":47,"min_child_samples":120,"reg_lambda":2.0,
  "reg_alpha":0.1,"learning_rate":0.05},
 {"num_leaves":31,"min_child_samples":150,"reg_lambda":4.0,
  "reg_alpha":0.3,"learning_rate":0.03},
]
for parameters in search_space:
 model = make_lgb(parameters, n_estimators=900)
 model.fit(X_train_encoded, y_train,
   eval_set=[(X_validation_encoded, y_validation)],
   eval_metric="binary_logloss",
   callbacks=[early_stopping(60, verbose=False)])
 probability = model.predict_proba(X_validation_encoded)[:, 1]
 score = f1_score(y_validation, probability >= .50, average="macro")
 record_trial(parameters, model.best_iteration_, score)
```

Question 5.1 - bounded LightGBM search on validation only

5.2 Validation-only selection and pre/post comparison (4 marks)

WHAT THE QUESTION ASKS

Use cross-validation or a strict validation set, never test, to select hyperparameters. Report the selected configuration and compare performance before and after optimisation.

Answer to 5.2

Trial 5 had the best structural validation macro-F1 at threshold 0.50: 0.600. It used 47 leaves, min_child_samples 120, learning rate 0.05, L1 0.1, L2 2.0 and 15 boosting rounds. A validation-only threshold search selected 0.65, where macro-F1 was 0.703. Base LightGBM scored 0.575 at threshold 0.50. The change from 0.575 to 0.703 includes both model settings and the threshold change, so it is not a pure model-structure gain.

ITEM	SELECTED VALUE
Leaves	47

Minimum child samples	120
Learning rate	0.05
L1 / L2	0.1 / 2.0
Boosting iterations	15
Decision threshold	0.65
Base validation macro-F1	0.575
Optimised validation macro-F1	0.703

Code used for 5.2

```
best_model = max(trials, key=lambda item: item["val_macro_f1"])
best_probability = best_model.predict_proba(X_validation_encoded)[:, 1]
threshold_rows = []
for threshold in np.arange(0.20, 0.801, 0.01):
 prediction = (best_probability >= threshold).astype("int8")
 threshold_rows.append({
  "threshold": threshold,
  "validation_macro_f1": f1_score(
    y_validation, prediction, average="macro"
  )
 })
selected = max(threshold_rows, key=lambda item: item["validation_macro_f1"])
# The locked test set is not read anywhere in this selection block.
```

Question 5.2 - validation-only threshold selection

5.3 One-time final test and practical meaning (2 marks)

WHAT THE QUESTION ASKS

Lock the entire pipeline, evaluate it once on test and explain whether the improvement is practically meaningful.

Answer to 5.3

We refitted the selected model on training plus validation for 15 boosting rounds. We then tested it once on 35,543 locked test cases using threshold 0.65. It achieved accuracy 0.727, macro-F1 0.698, ROC-AUC 0.791, PR-AUC 0.789, at-risk precision 0.845 and at-risk recall 0.469. The bootstrap 95% interval for macro-F1 was 0.692-0.702.

Compared with base LightGBM, test macro-F1 changed only from 0.6969 to 0.6976, a gain of +0.0006. This is too small to matter in practice. The main change is the trade-off: precision increased from 0.620 to 0.845, while recall fell from 0.810 to 0.469. Threshold 0.65 therefore suits a small, high-precision review queue.

Code used for 5.3

```
# Final configuration is fixed before this block.
X_train_validation = pd.concat([X_train, X_validation])
y_train_validation = np.r_[y_train, y_validation]
final_maps = fit_category_maps(X_train_validation)
X_tv = encode_for_lightgbm(X_train_validation, final_maps)
X_test_final = encode_for_lightgbm(X_test, final_maps)
final_model = make_lgb(best_parameters, n_estimators=15)
final_model.fit(X_tv, y_train_validation,
 categorical_feature=list(range(len(CAT))))
test_probability = final_model.predict_proba(X_test_final)[:, 1]
test_prediction = (test_probability >= 0.65).astype("int8")
final_test_macro_f1 = f1_score(
 y_test, test_prediction, average="macro"
)
```

Question 5.3 - locked final refit and one-time test evaluation

KEY POINT

The optimisation used validation data and can be repeated. It produced a high-precision threshold, but test macro-F1 barely changed from base LightGBM.

INTRODUCTION TO DATA SCIENCE

QUESTION 6 - REVISED ANSWER

Ablation and McNemar Validation

Separate answers for Sub-questions 6.1 and 6.2

DOCUMENT STRUCTURE

Each sub-question is answered independently under its exact number. Applicable code is placed directly under the step it supports. Any item not yet completed is labelled as a placeholder rather than presented as finished.

6.1 Controlled ablation experiments (5 marks)

WHAT THE QUESTION ASKS

Conduct at least three one change at a time experiments while keeping the split, seed and evaluation protocol fixed. Report performance deltas and interpret what the system depends on.

Answer to 6.1

We ran four controlled ablation tests. Every test used the same training and validation dates, seed 42, LightGBM settings and threshold 0.65. Only one feature group or class-weighting setting changed at a time.

EXPERIMENT	VAL MACRO-F1	DELTA
Full optimized pipeline	0.703	+0.000
Remove Work_Center	0.654	-0.049
Remove location group	0.700	-0.003
Remove temporal group	0.673	-0.030
No class weighting	0.607	-0.096

Removing class weighting caused the biggest drop (-0.096). Removing Work_Center reduced macro-F1 by 0.049, and removing time features reduced it by 0.030. Removing Sub_Council, Ward and Suburb together caused only a small drop (-0.003). This shows that weighting, work-centre information and time are important for prediction. It also suggests that a version without detailed location may keep similar overall performance. These results show prediction dependence, not cause and effect.

Code used for 6.1

```
ablations = [
 ("Remove Work_Center", ["Work_Center"], False),
 ("Remove location group", ["Sub_Council","Ward","Suburb"], False),
 ("Remove temporal group",
  ["day_of_week","day_index","day_of_month","is_weekend"], False),
 ("No class weighting", [], True),
]
full_score = validation_macro_f1(full_model, threshold=0.65)
for name, dropped_columns, no_weight in ablations:
 categorical = [c for c in CAT if c not in dropped_columns]
 numeric = [c for c in NUM if c not in dropped_columns]
 parameters = dict(best_parameters)
```

```
parameters["class_weight"] = None if no_weight else "balanced"
ablated_model = fit_lightgbm(
 X_train[categorical + numeric], y_train, parameters,
 seed=42, iterations=15
)
score = macro_f1_on_validation(ablated_model, threshold=0.65)
ablation_log.append({"name":name,"score":score,
                     "delta":score-full_score})
```

Question 6.1 - one change at a time ablation loop

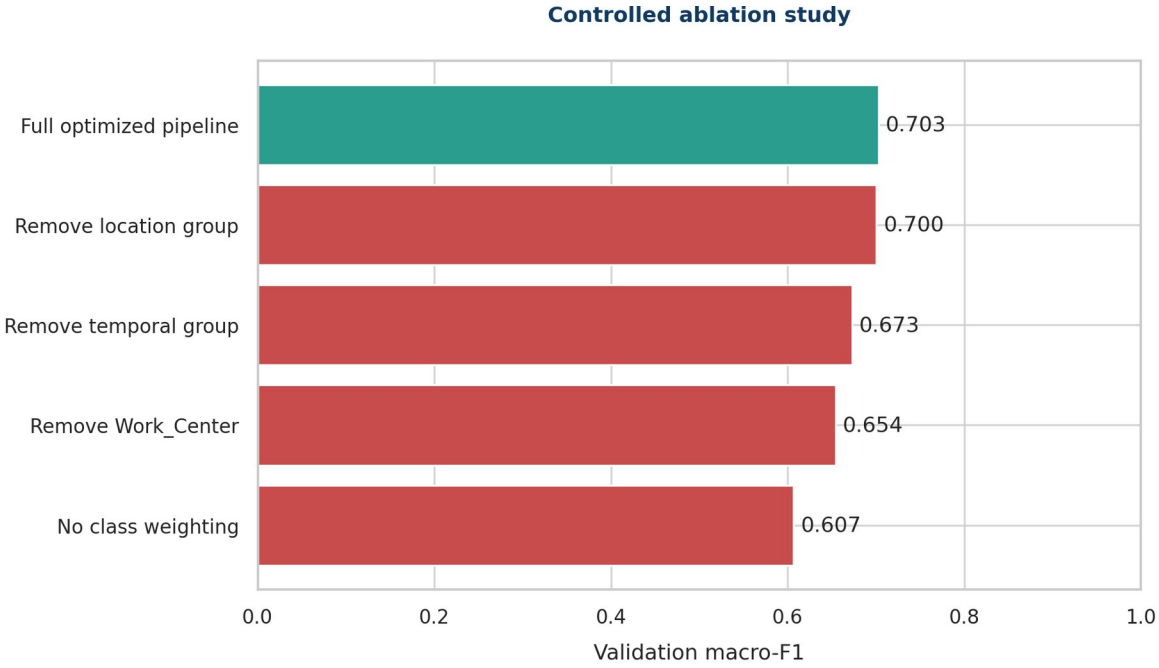


Figure 6.1. Controlled validation ablations.

6.2 McNemar's paired validation (5 marks)

WHAT THE QUESTION ASKS

Compare the optimized champion with the strongest comparator on the same test cases. Show the 2x2 correctness table, H0/H1, alpha=0.05, statistic/exact p-value, and distinguish statistical from practical significance.

Answer to 6.2

We compared the optimized LightGBM with the dense MLP, which was the best unoptimised validation model. Both models predicted the same 35,543 locked test cases. The paired 2x2 table compares whether each model was correct.

	DENSE MLP CORRECT	DENSE MLP WRONG
Champion correct	17,090	8,758
Champion wrong	5,734	3,961

- H0: the probability that only the champion is correct equals the probability that only the dense MLP is correct (b = c).
- H1: the two discordant probabilities differ (b != c).
- Significance level: alpha = 0.05.

The models disagreed on 14,492 cases. The corrected statistic was 630.59 with p=3.72e-139. The exact binomial p-value was 3.63e-140. Both are below 0.05, so we reject H0. LightGBM was correct on 3,024 more disagreement cases (8,758 versus 5,734).

McNemar’s test evaluates paired correctness on the same cases: it tests whether the two models have equal probabilities of being uniquely correct on their discordant predictions. It does not test macro-F1, the difference between macro-F1 scores, or any class-specific metric. The paired-correctness difference is statistically significant, but practical model choice still requires the separately reported macro-F1, precision and recall results. The dense MLP found more at-risk cases, while LightGBM created a smaller queue with higher precision.

Code used for 6.2

```
champion_correct = champion_prediction == y_test
comparator_correct = mlp_prediction == y_test
both_correct = int(np.sum(champion_correct & comparator_correct))
champion_only = int(np.sum(champion_correct & ~comparator_correct))
comparator_only = int(np.sum(~champion_correct & comparator_correct))
both_wrong = int(np.sum(~champion_correct & ~comparator_correct))
correctness_table = [
 [both_correct, champion_only],
 [comparator_only, both_wrong],
]
discordant = champion_only + comparator_only
result = mcnemar(correctness_table,
 exact=(discordant < 25), correction=(discordant >= 25))
exact_p = binomtest(
 min(champion_only, comparator_only), n=discordant,
 p=0.5, alternative="two-sided"
).pvalue
```

Question 6.2 - paired correctness table and McNemar's test

KEY POINT
McNemar’s test uses the paired correct/incorrect outcomes for the same held-out cases and tests the imbalance between the two discordant cells. It does not test macro-F1. Macro-F1, precision and recall remain separate descriptive performance measures used for practical interpretation.

The ablations show which parts affect performance. McNemar’s test shows that LightGBM wins more paired disagreements than the dense MLP, but the recall trade-off remains.

INTRODUCTION TO DATA SCIENCE

QUESTION 7 - REVISED ANSWER

User-Interface Deployment

Separate answers for Sub-questions 7.1, 7.2, 7.3 and 7.4

DOCUMENT STRUCTURE

Each sub-question is answered independently under its exact number. Applicable code is placed directly under the step it supports. Any item not yet completed is labelled as a placeholder rather than presented as finished.

7.1 Deploy the final optimized model through a working interface (2 marks)

WHAT THE QUESTION ASKS

Deploy the final optimized model through Gradio, Streamlit, Flask, FastAPI with a web front end, or another approved framework. A notebook cell alone is not a user interface.

Answer to 7.1

We built a Streamlit web interface for the saved optimized LightGBM model. It loads the trained category maps, feature lists and threshold. A local smoke test returned HTTP 200, showing that the app starts correctly. The app can be reproduced locally. No public link is claimed.

Code used for 7.1

```
st.set_page_config(page_title="Service Request Risk Triage", page_icon="", layout="centered")
BUNDLE_PATH = Path(__file__).parent / "model_outputs" / "service_request_risk_model.joblib"

@st.cache_resource
def load_bundle():
    return joblib.load(BUNDLE_PATH)

bundle = load_bundle()
model = bundle["model"]
category_maps = bundle["category_maps"]
cat_features = bundle["categorical_features"]
num_features = bundle["numeric_features"]
threshold = float(bundle["threshold"])

st.title("Municipal Service Request - Seven-Day Risk Triage")
st.caption("Decision support only: predicts risk that a request will not be completed within seven calendar days.")
```

Question 7.1 - Streamlit setup and saved-bundle loading

7.2 Valid input, complete pipeline and understandable output (2 marks)

WHAT THE QUESTION ASKS

Allow a user to enter/upload valid input, run the complete saved preprocessing-and-model pipeline, and display predicted class, probability/confidence and a brief explanation.

Answer to 7.2

The form accepts creation date, sub-council, ward, suburb, complaint type, work centre and coordinate quality. It creates the same date and quality features used during training, maps categories with the saved training maps, calculates the probability and applies threshold 0.65. It shows the class, probability and the three strongest model signals. These signals explain the model score but do not prove cause and effect.

Code used for 7.2

```
with st.form("risk_form"):
    created_date = st.date_input("Request creation date", value=dt.date(2026, 5, 30))
    sub_council = st.selectbox("Sub-council", sorted(category_maps["Sub_Council"].keys()))
    ward = st.selectbox("Ward", sorted(category_maps["Ward"].keys()))
    suburb = st.selectbox("Suburb", sorted(category_maps["Suburb"].keys()))
    complaint = st.selectbox("Complaint type", sorted(category_maps["C3_Complaint_Type"].keys()))
    work_center = st.selectbox("Work centre", sorted(category_maps["Work_Center"].keys()))
    coordinate_state = st.radio("Coordinate quality", ["Available", "Missing", "Recorded as 0,0"], horizontal=True)
    submitted = st.form_submit_button("Assess risk", type="primary")

if submitted:
    errors = []
    if not complaint or complaint == "Missing":
        errors.append("Complaint type is required.")
    if created_date < dt.date(2026, 4, 1):
        errors.append("Creation date is outside the supported period (from 1 April 2026).")
    if errors:
        for message in errors:
            st.error(message)
    else:
        row = {
            "Sub_Council": sub_council,
            "Ward": ward,
            "Suburb": suburb,
            "C3_Complaint_Type": complaint,
            "Work_Center": work_center,
            "day_of_week": created_date.strftime("%A"),
            "day_index": float((created_date - dt.date(2026, 4, 1)).days),
            "day_of_month": float(created_date.day),
            "is_weekend": float(created_date.weekday() >= 5),
            "coord_missing": float(coordinate_state == "Missing"),
            "coord_zero": float(coordinate_state == "Recorded as 0,0"),
        }
        encoded = np.zeros((1, len(cat_features) + len(num_features)), dtype=np.float32)
        for j, feature in enumerate(cat_features):
            encoded[0, j] = category_maps[feature].get(str(row[feature]), 0)
        for j, feature in enumerate(num_features, start=len(cat_features)):
            encoded[0, j] = row[feature]
        probability = float(model.predict_proba(encoded)[0, 1])
        predicted_at_risk = probability >= threshold
        if predicted_at_risk:
            st.error("High-priority review: at risk of exceeding seven days")
        else:
            st.success("Routine monitoring: likely to complete within seven days")
        st.metric("Predicted non-completion risk", f"{probability:.1%}", help=f"Decision threshold: {threshold:.0%}")
        contributions = model.booster_.predict(encoded, pred_contrib=True)[0][:-1]
        top = sorted(zip(cat_features + num_features, contributions), key=lambda x: abs(x[1]), reverse=True)[:3]
        st.subheader("Main model signals")
        for feature, value in top:
            direction = "increased" if value > 0 else "reduced"
            st.write(f"- **{feature}** {direction} the model score (contribution {value:+.3f}).")
        st.info("Use this result to support triage, not to deny, close or deprioritise a request. A human operator must
consider service criticality, vulnerability and current capacity.")
```

Question 7.2 - input, feature construction, inference and explanation

7.3 Validation, helpful errors, guidance and responsible use (1 mark)

WHAT THE QUESTION ASKS

Include input validation, helpful errors, class/feature guidance, a limitation/use warning, and no exposed credentials or personal data.

Answer to 7.3

Complaint type is required, and dates before 1 April 2026 show an error. Drop-down lists limit category values. The form does not accept names, phone numbers, emails, free text, exact addresses or login details. A warning says that the score must not be used to deny, close or delay a request. A person must still consider urgency, vulnerability and available staff.

Code used for 7.3

```
with st.expander("Limitations and responsible use"):
    st.write("The model was trained on a two-month historical extract. Performance can drift as workloads and workflows change.
Subgroup recall varied materially by sub-council, so monthly error-rate monitoring and human review are required. Do not enter
names, contact details, free-text descriptions or exact residential addresses.")
```

Question 7.3 - limitations and use warning

7.4 Launch instructions and demonstrated cases (2 marks)

WHAT THE QUESTION ASKS

Provide use instructions and either a stable demonstration link or a locally reproducible launch command. Demonstrate at least three valid cases and one invalid-input case.

Answer to 7.4

The local app was completed and smoke-tested. After installing the listed packages, run it with streamlit run streamlit_app.py. A public URL is still a clearly marked placeholder.

```
pip install -r requirements.txt
python -m streamlit run 03_Code/app.py
streamlit run streamlit_app.py
```

Question 7.4 - locally reproducible launch command

CASE	DATE	COMPLAINT	RISK	PREDICTION	OBSERVED / ERROR
1	2026-05-23	Suspect Tampering	79.7%	At risk of exceeding 7 days	Observed at risk; verified prediction passed
2	2026-05-28	Meter: Damaged / Faulty	68.2%	At risk of exceeding 7 days	Observed at risk; verified prediction passed
3	2026-05-30	MTR: Customer Reconnection Request	51.6%	Likely completed within 7 days	Observed within 7 days; verified prediction passed
Invalid	-	Blank complaint type	-	Validation error	Complaint type is required; verified validation passed

STATUS / PLACEHOLDERS
LOCAL LAUNCH: VERIFIED with the documented command; the Streamlit health endpoint returned HTTP 200. DEMONSTRATION: three valid held-out cases and one invalid blank-complaint case are recorded in 05_Results/interface_ui_tests.json. PUBLIC APP LINK: [NOT CREATED / NOT VERIFIED]. PRESENTATION RECORDING LINK: [NOT CREATED / NOT VERIFIED]. Add real lecturer-accessible links only after publication or recording; do not fabricate them.

PUBLIC DEMONSTRATION LINK: [NOT YET CREATED]. Use the locally reproducible launch command or add a real lecturer-accessible link/recording before final submission. Do not replace this placeholder with a fabricated URL.

Smoke-test code used for 7.4

```
streamlit run streamlit_app.py --server.headless true --server.port 8765 &
curl -s -o /dev/null -w "%{http_code}" http://127.0.0.1:8765/
# Verified result: HTTP 200
```

Question 7.4 - local server smoke test

KEY POINT

The Streamlit app loads the saved pipeline, checks inputs, shows the class, probability and explanation, and states its limits. It can be launched locally. A public link is still unfinished.

INTRODUCTION TO DATA SCIENCE

QUESTION 8 - REVISED ANSWER

Recommendations, Reproducibility and Team Defence

Separate answers for Sub-questions 8.1, 8.2 and 8.3

DOCUMENT STRUCTURE

Each sub-question is answered independently under its exact number. Applicable code is placed directly under the step it supports. Any item not yet completed is labelled as a placeholder rather than presented as finished.

8.1 Recommendation, limitations, uncertainty, fairness and monitoring (2 marks)

WHAT THE QUESTION ASKS

Translate findings into a realistic stakeholder recommendation. State limitations, uncertainty, fairness findings, monitoring needs and conditions under which the model should not be used.

Answer to 8.1

Run an 8-12 week shadow-mode pilot. The model should add cases to a human review queue without changing anyone's right to service or closing or delaying requests automatically. Emergency, health, safety and vulnerability rules must always override the score.

REVIEW SHARE	FLAGGED	RISK CAPTURE	PRECISION
10%	3,554	21.7%	95.8%
20%	7,108	40.2%	88.8%
25%	8,885	47.7%	84.2%
30%	10,662	54.1%	79.7%
40%	14,217	65.1%	71.8%
50%	17,771	73.4%	64.9%

In the test data, a 20% review queue found 40.2% of at-risk cases with 88.8% precision. This supports testing the idea in a pilot, but it does not guarantee future benefit. The final model's test macro-F1 was 0.698 with a bootstrap 95% interval of 0.692-0.702. Across 21 sub-council groups with at least 200 test rows, recall ranged from 13.1% to 58.1% and false-positive rate from 0.0% to 13.9%. The model must not make decisions by itself, be used outside similar Cape Town processes, rank staff, or be used after major process changes without new testing.

Code used for 8.1

```
# Capacity scenario: rank the locked test cases by risk score.
```

```
order = np.argsort(-champion_probability)
for review_share in [0.10,0.20,0.25,0.30,0.40,0.50]:
 n = int(len(y_test) * review_share)
 reviewed = order[:n]
 capture = y_test[reviewed].sum() / y_test.sum()
 precision = y_test[reviewed].mean()
 capacity_rows.append((review_share, n, capture, precision))

# Subgroup audit; suppress very small groups.
for group, index in test_frame.groupby("Sub_Council").groups.items():
 if len(index) < 200:
  continue
 subgroup_recall = recall_score(y_test[index], prediction[index])
 subgroup_fpr = false_positive_rate(y_test[index], prediction[index])
```

Question 8.1 - capacity scenario and subgroup audit

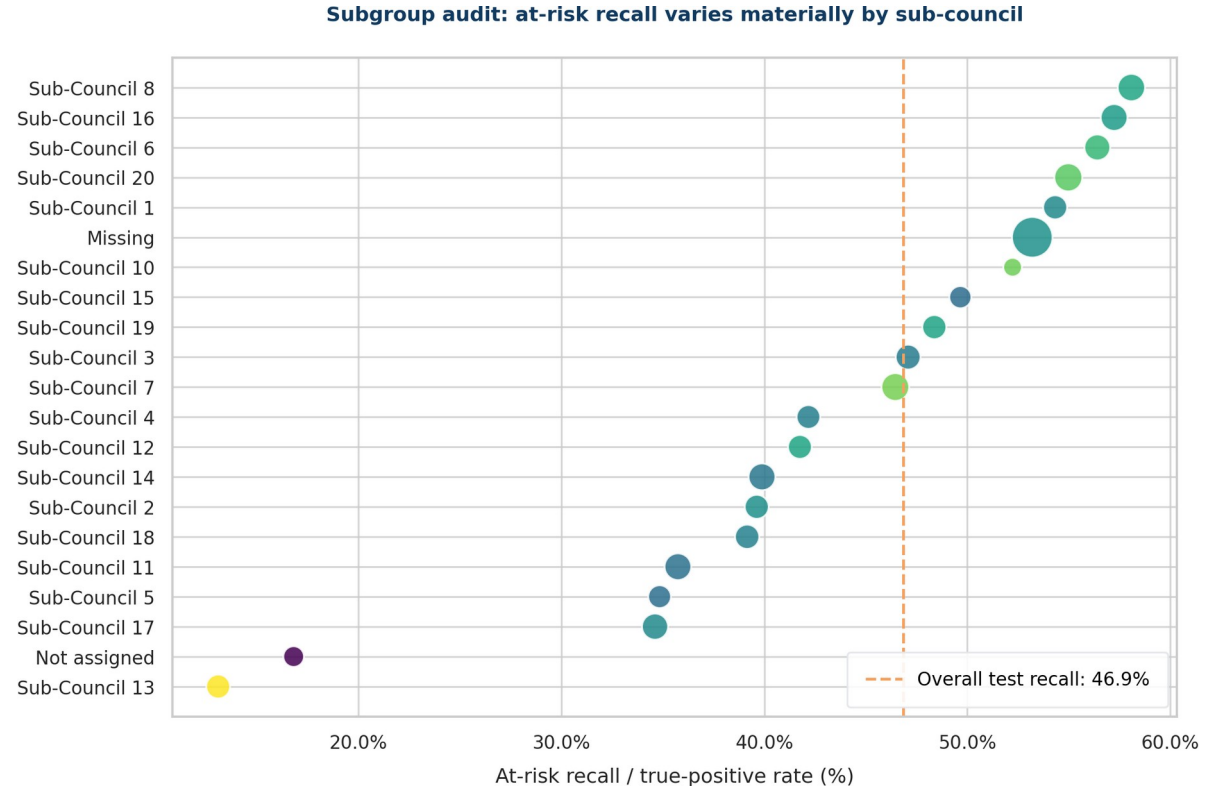


Figure 8.1. At-risk recall differs clearly by sub-council.

8.2 Reproducible repository/package (3 marks)

WHAT THE QUESTION ASKS

Submit a reproducible repository/package with README, environment/requirements, fixed seeds, execution order, source links, data dictionary, saved model/pipeline and UI code. Do not redistribute data if its licence forbids it.

Answer to 8.2

The completed manual-upload package contains the final README, exact pinned requirements, seed 42, ordered preparation, modelling and evaluation scripts, a reproduction notebook, saved model bundles, held-out predictions, optimisation, ablation and learning-curve evidence, figures/results, the Streamlit application, a data dictionary, source/licence notes, the assignment report, contribution-statement template and upload instructions. The exact target URL supplied for upload is https://github.com/mahlatseselowa/IDA117V.git; public accessibility and the final commit hash have not been verified in this work and must be confirmed after upload to that exact repository. Local Streamlit launch and test evidence are verified; no public app link exists yet. The portal item was recorded as Public with a Custom License, but the exact item-level terms, source-to-file chain of custody and redistribution approval are not evidenced, so the raw dataset is excluded.

PACKAGE ITEM	STATUS / REQUIRED CONTENT
README.md	COMPLETED IN PACKAGE: setup, exact execution order, expected outputs, ethical limits, verification and launch instructions are documented.
requirements.txt	COMPLETED IN PACKAGE: all direct runtime dependencies are pinned to exact versions used for the verified run.
Source code	COMPLETED IN PACKAGE: ordered preparation, modelling, evaluation, saved-inference, UI and verification scripts plus the reproduction notebook.
Saved evidence	COMPLETED IN PACKAGE: saved deployment model, comparison bundle, held-out predictions, trials, ablations, learning curves, metrics, figures and UI-test evidence.
Data dictionary / card	COMPLETED IN PACKAGE: fields, target, missingness, risks, leakage controls and source-snapshot notes are documented.
Dataset licence	RECORDED BUT NOT CLEARED: the assignment records Public/Custom License metadata, but exact item-level terms, chain of custody and written redistribution approval are not evidenced. Raw data is excluded and approval is not claimed.
Repository URL / commit	TARGET URL SUPPLIED: https://github.com/mahlatseselowa/IDA117V.git. PACKAGE: completed and verified locally for manual upload. PUBLIC ACCESSIBILITY: unverified. FINAL COMMIT HASH: pending upload/commit to this exact repository. PUBLIC STREAMLIT LINK: not created. LOCAL DEPLOYMENT: verified.

Code used for 8.2

```
# Reproducibility constants
SEED = 42
random.seed(SEED)
np.random.seed(SEED)

# Save the complete fitted inference bundle.
joblib.dump({
 "model": final_model,
 "category_maps": final_category_maps,
 "categorical_features": CAT,
 "numeric_features": NUM,
 "threshold": 0.65,
 "target_definition": "1 = not completed within seven days",
 "random_seed": SEED,
 "best_parameters": best_parameters,
 "best_iteration": 15,
}, "model_outputs/service_request_risk_model.joblib", compress=3)

# Save exact paired evidence for future audit.
np.savez_compressed(
 "model_outputs/heldout_predictions.npz",
 y_true=y_test,
 champion_probability=champion_probability,
 comparator_probability=mlp_probability,
)
```

Question 8.2 - fixed seeds, saved pipeline and test evidence

8.3 Presentation, demonstration and signed contribution statement (2 marks)

WHAT THE QUESTION ASKS

Deliver a concise team presentation and live/recorded interface demonstration. Defend modelling choices and include a signed contribution statement; individual marks may be moderated.

Answer to 8.3

A 10-12 minute presentation plan and verified local Streamlit demonstration evidence are included. Three valid cases, one invalid case and HTTP 200 are recorded. McNemar is presented as a paired-correctness test, not a macro-F1 test. Public app/recording links do not exist in the supplied evidence. The contribution template is blank and unsigned.

TIME	PRESENTATION CONTENT
0:00-1:00	Problem, stakeholder and seven-day target
1:00-2:30	Data provenance, audit, leakage and ethics
2:30-4:00	EDA and chronological split
4:00-6:00	Model set, metrics and overfitting
6:00-7:30	Optimization, ablation and McNemar
7:30-9:30	Interface: three valid and one invalid case
9:30-12:00	Recommendation, fairness, limitations and questions

STATUS / REQUIRED COMPLETION
PLAN AND LOCAL DEMO: COMPLETED (three valid, one invalid and HTTP 200). PUBLIC APP LINK: [NOT CREATED / NOT VERIFIED]. RECORDING LINK: [NOT CREATED / NOT VERIFIED]. CONTRIBUTION TEMPLATE: included but unsigned; enter only real member details and signatures.

PRESENTATION/RECORDING: [NOT YET DELIVERED]. CONTRIBUTION STATEMENT: [INSERT REAL MEMBER NAMES, STUDENT NUMBERS, SPECIFIC CONTRIBUTIONS AND SIGNATURES]. Do not invent member contributions or signatures.

Code supporting the 8.3 interface demonstration

```
# Optional local demonstration logging - no personal data.
demo_cases = [low_risk_case, borderline_case, high_risk_case]
for case in demo_cases:
 result = run_saved_pipeline(case)
 print({
  "predicted_class": result.predicted_class,
  "risk_probability": result.probability,
  "responsible_use_notice_shown": True,
 })
try:
 run_saved_pipeline(invalid_blank_complaint_case)
except ValueError as error:
 print(str(error))  # Expected: Complaint type is required.
```

Question 8.3 - demonstration cases and expected invalid-input handling

KEY POINT
The manual-upload package and local demo are verified. Public repository access, final commit, data redistribution, lecturer architecture approval, public links and signed contributions remain unverified or outstanding.

The evidence supports a careful pilot with human control and a reproducible public-repository package. Manual GitHub upload, the final target-repository commit hash, the presentation or recording, and signed contributions remain submission steps.

Compilation source ledger

INCLUDED SOURCE	SIZE	SHA-256 PREFIX
IDA117V_Question_1_Separate_Subquestion_Answers_Verified_Evidence.docx	314,940 bytes	c047b76f955fc346...
IDA117V_Question_2_Separate_Subquestion_Answers_With_Code.docx	1,406,627 bytes	0d80558d2584e5f8...
IDA117V_Question_3_Separate_Subquestion_Answers_With_Code.docx	47,289 bytes	fe125638aa82cde1...
IDA117V_Question_4_Separate_Subquestion_Answers_With_Code.docx	1,072,177 bytes	d0091f6a9bfed10a...
IDA117V_Question_5_Separate_Subquestion_Answers_With_Code.docx	166,103 bytes	2fe526c713800d99...
IDA117V_Question_6_Separate_Subquestion_Answers_With_Code.docx	110,327 bytes	8124a55dc5c1fa78...
IDA117V_Question_7_Separate_Subquestion_Answers_With_Code.docx	43,231 bytes	391fec130812877e...
IDA117V_Question_8_Separate_Subquestion_Answers_With_Code.docx	222,876 bytes	08d4ed53ad3b2ee6...

SOURCE CONTROL

Only the eight files listed above were merged into this final assignment. The ledger supports traceability and confirms the compilation boundary.